2. Deployment of the Core Infrastructure

Lernziele:

- Sicherheitsprinzipien und Architekturentwurf
- WLAN und WLAN-Access Point
- DHCP und DNS Funktionsweise
- Firewall nftables
- Routing
- NAT
- Raspberry PI Pico: Installation und Programmierung mit Micro Python

Überblick:

2.1 Hardware

2.2 Architektur

2.3 Raspberry PI OS Installation

2.4 WLAN-AP und Router

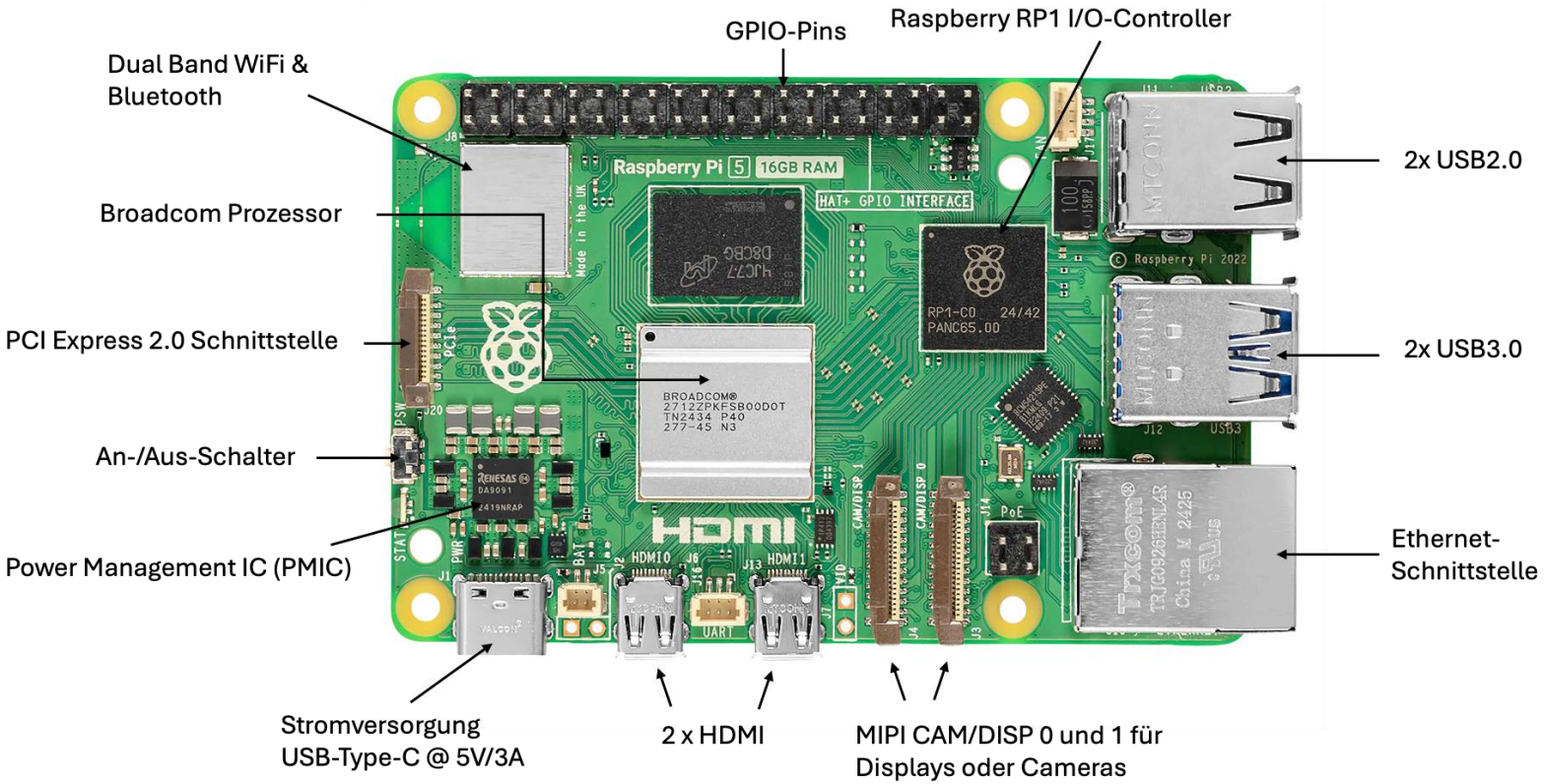
2.5 Firewall

2.6 Backend-Server Integration

2.7 Microcontroller Integration

2.8 Projektumsetzung

2.1 Hardware


GPIO-Pins
Raspberry RP1 I/O-Controller
Dual Band WiFi & Bluetooth
2x USB2.0
Broadcom Prozessor
PCI Express 2.0 Schnittstelle
2x USB3.0
An-/Aus-Schalter
Power Management IC (PMIC)
Ethernet-Schnittstelle
Stromversorgung USB-Type-C @ 5V/3A
2 x HDMI
MIPI CAM/DISP 0 und 1 für Displays oder Cameras

Netzwerke & Systeme

- Für dieses Hochschulprojekt stehen als Rechnerknoten 2 Raspberry PIs zur Verfügung:

  1x Raspberry PI für den Betrieb eines WLAN-Router inklusive Firewall.

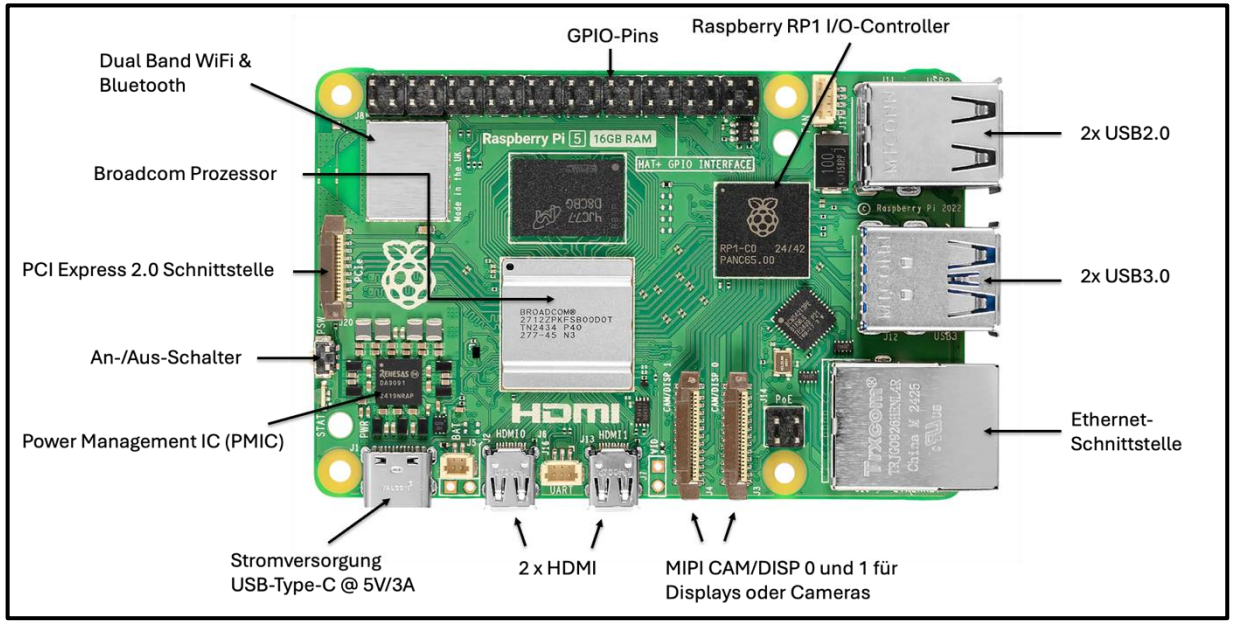
  1x Raspberry PI für den Betrieb eines Backend-Servers.
- Zum Auslesen und Übertragen der Sensor- und Aktor-Daten steht Ihnen als Mikrocontroller (MCU)

  1 x Raspberry Pi Pico WH

  zur Verfügung
- Der Rasberry PI Pico WH besitzt ein WLAN-Modul (2,4-GHz-Single-Band-Schnittstelle - 802.11n) und kann so in das WLAN-Netzwerk des WLAN-Routers integriert werden.
- Als Software für die benötigte Infrastruktur werden durchgängig Open Source Systeme verwendet.

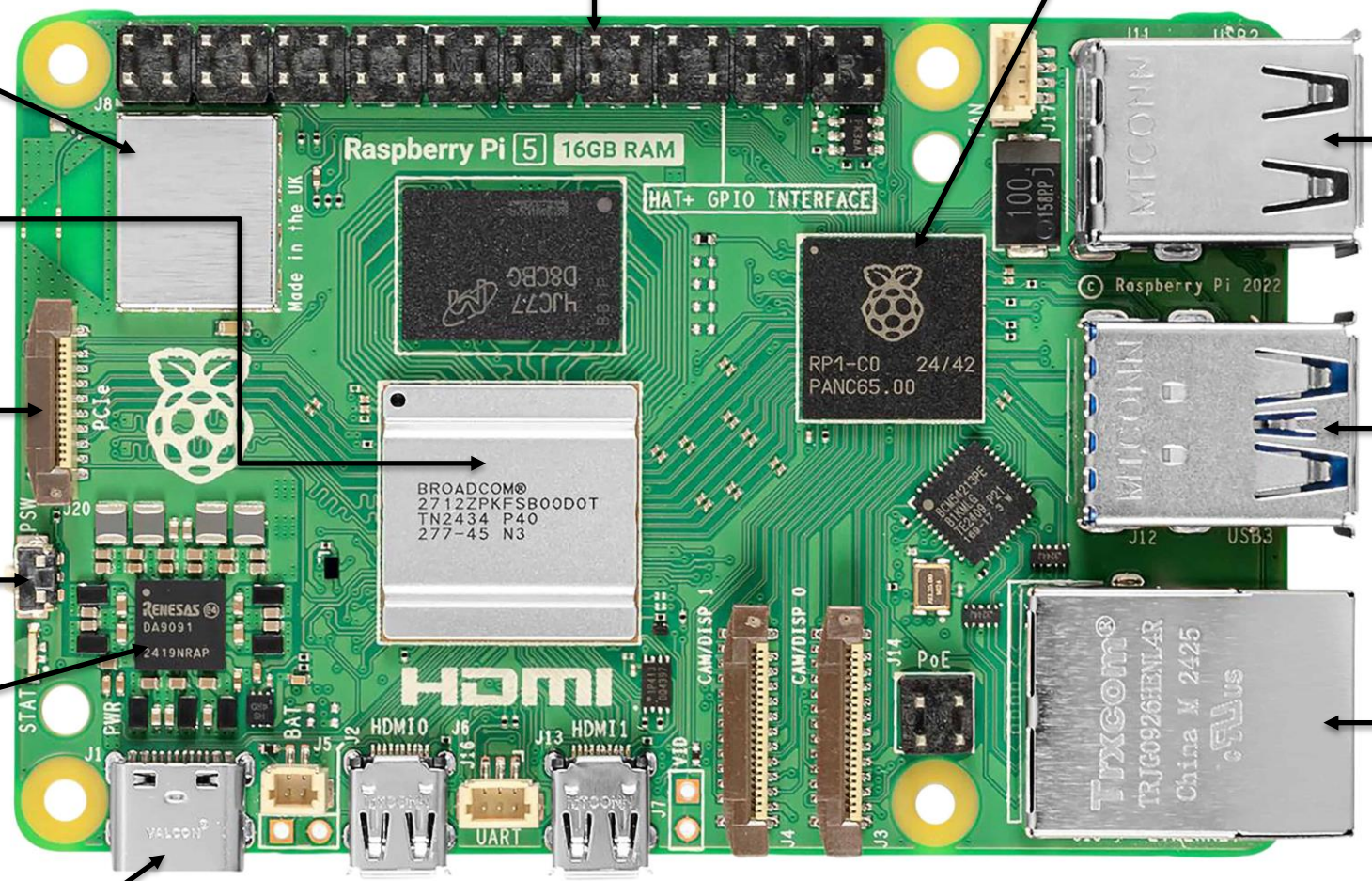
Rechnerkoten: Raspberry PI

An-zahl	Hardware	Einsatz
1	Raspberry Pi 5 , 16GB RAM	Backend- und Datenbankserver SW: Raspberry PI OS, docker,
1	Raspberry Pi 5 , 2/4GB RAM	WLAN-AP, Router, Firewall SW: Raspberry PI OS, nftables, dnsmasq (DNS- und DHCP)
2	Raspberry Pi 27W USB-C Power Supply	Stromversorgung
2	*12V Aktiver Lüfter für Rasberry PI 5*	*Kühlung*
1	Micro SD 32GB	Permanenter Flash-Speicher für DB-Server
1	Micro SD 32GB	Permanenter Flash-Speicher für Wireless Router



- Quad-Core ARM Cortex-A76 Prozessor 64 Bit
- 1 x Ethernet
- Wi-Fi 802.11ac (2.4 GHz / 5.0 GHz)-Adapter
- Bluetooth 5.0 / Bluetooth Low Energy (BLE) - Adapter
- SDRAM (synchrones DRAM) für effiziente Leistung
- 2 x Micro-HDMI für Dual-Display-Option
- 2 x MIPI-CAM/DISP für Camera und Display

Raspberry Pi 5



Dual Band WiFi & Bluetooth

GPIO-Pins

Raspberry RP1 I/O-Controller

2x USB2.0

Broadcom Prozessor

PCI Express 2.0 Schnittstelle

2x USB3.0

An-/Aus-Schalter

Power Management IC (PMIC)

Ethernet-Schnittstelle

Stromversorgung USB-Type-C @ 5V/3A

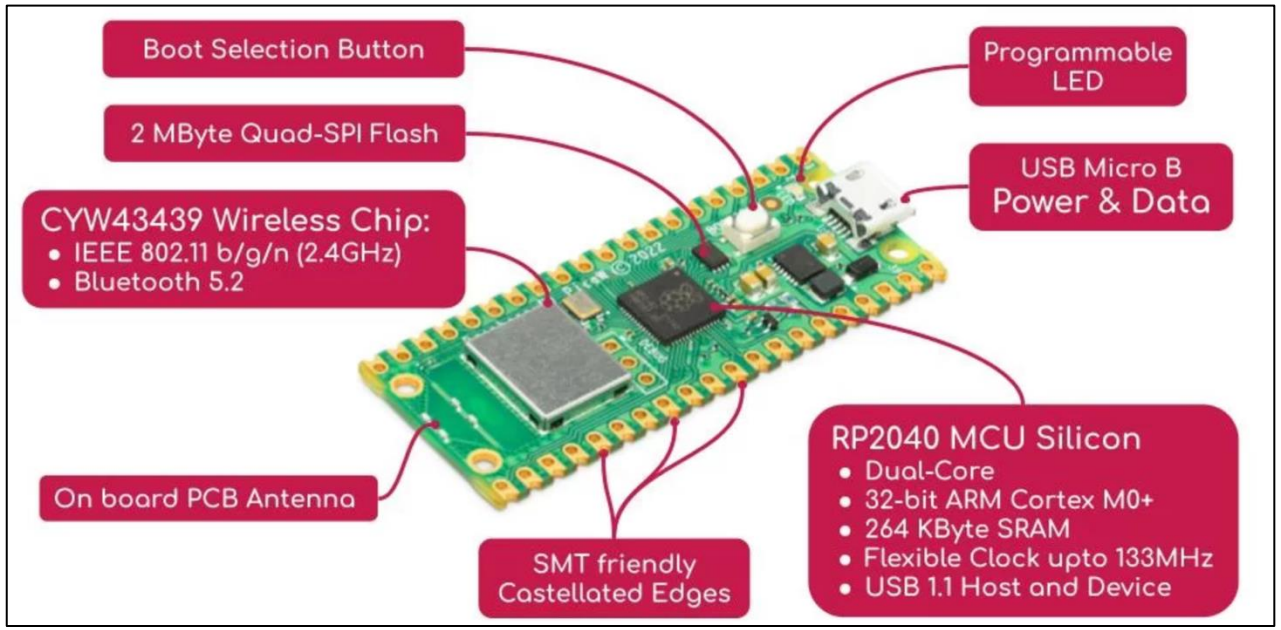
2 x HDMI

2 x MIPI CAM/DISP 0 und 1 für Displays oder Cameras

Raspberry Pi Pico W

- Ein **Raspberry Pi Pico W** ist ein Mikrocontroller-Board, das speziell dafür ausgelegt ist Sensoren auszulesen und Aktoren zu steuern.
- Ein Mikrocontrollerboard enthält auf seiner Platine typischerweise

Komponente	Beschreibung
Mikrocontroller	Dual-Core ARM Prozessor, 256kB RAM
Permanenter Speicher	2MB Flash-Speicher, Quad-SPI (Serial Peripheral Interface) mit 4 Datenleitungen
Schnittstellen	GPIO für die Steuerung von Sensoren, Aktoren
USB-Anschluss	Stromversorgung und für die Datenübertragung.
WLAN-Adapter	WLAN-Anbindung

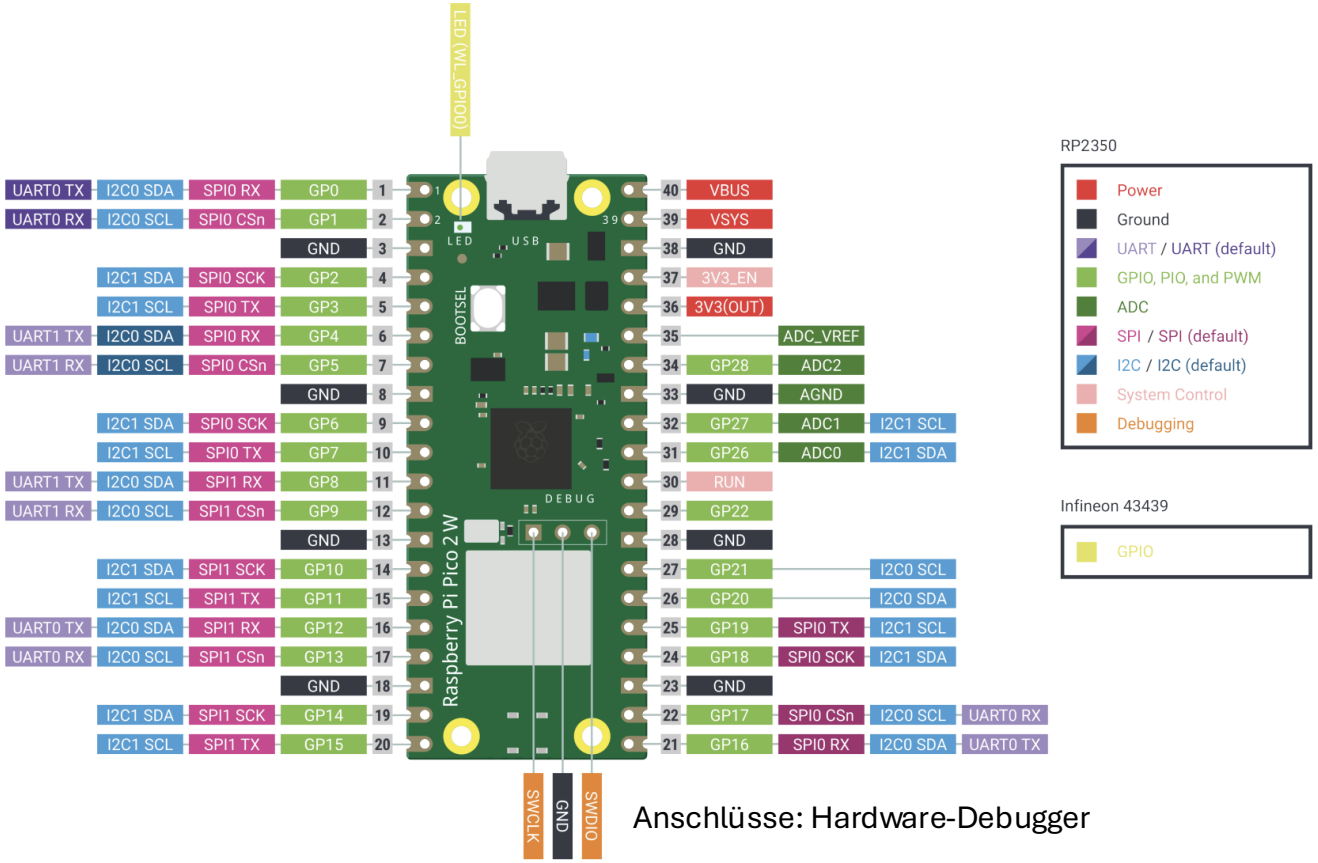


- **Software**
  - Beim Pico muss kein Betriebssystem installiert werden.
  - Die Installation erfolgt durch das Kopieren einer uf2-Datei (UF: USB Flashing Format) auf den Flash-Speicher

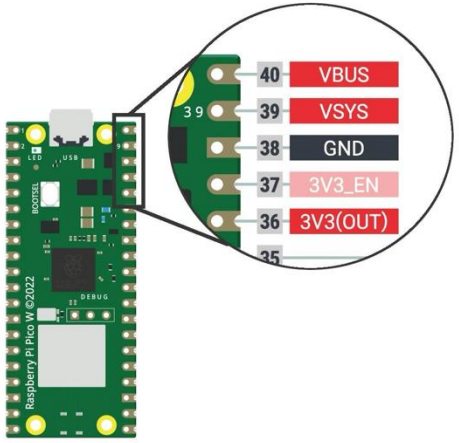
"Flash"-Speicher: Blitz-Speicher, lässt sich schnell löschen.

Raspberry Pi Pico W

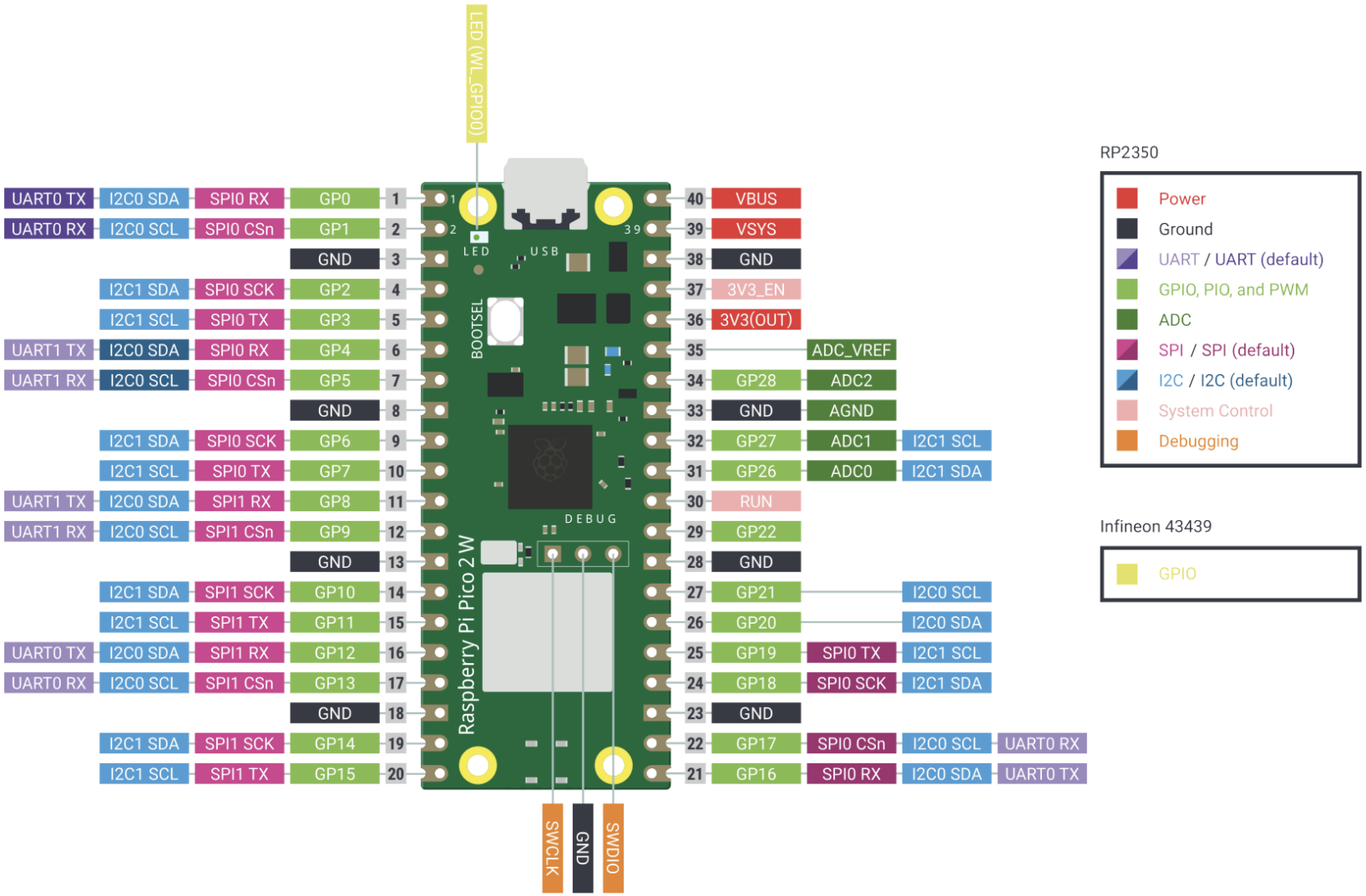
- Der **Raspberry Pi Pico** besitzt sogenannte GPIO-Pins, kurz für General Purpose Input/Output.
  - GPIOs sind vielseitige Schnittstellen, die es ermöglichen, Aktoren zu steuern und Signale von Sensoren zu empfangen.
  - Die Pins können als Ein- oder Ausgänge konfiguriert werden und arbeiten mit 3,3V.
  - Einsatzgebiete: Sensoren auslesen und Aktoren steuern.
- Beim Raspberry Pi Pico können Sensoren/Aktoren je nach Typ über die GPIO PINs **3V3(Out)** oder **VBUS** mit Gleichspannung versorgt werden
  - 3V3(Out): liefert 3,3V als Ausgangsspannung; für die meisten Sensoren der Standard.
  - 3V3_EN: enable PIN für den 3,3V Ausgang.
  - VBUS: liefert 5V als Ausgangsspannung
  - VSYS: Spannungsversorgung für den PICO



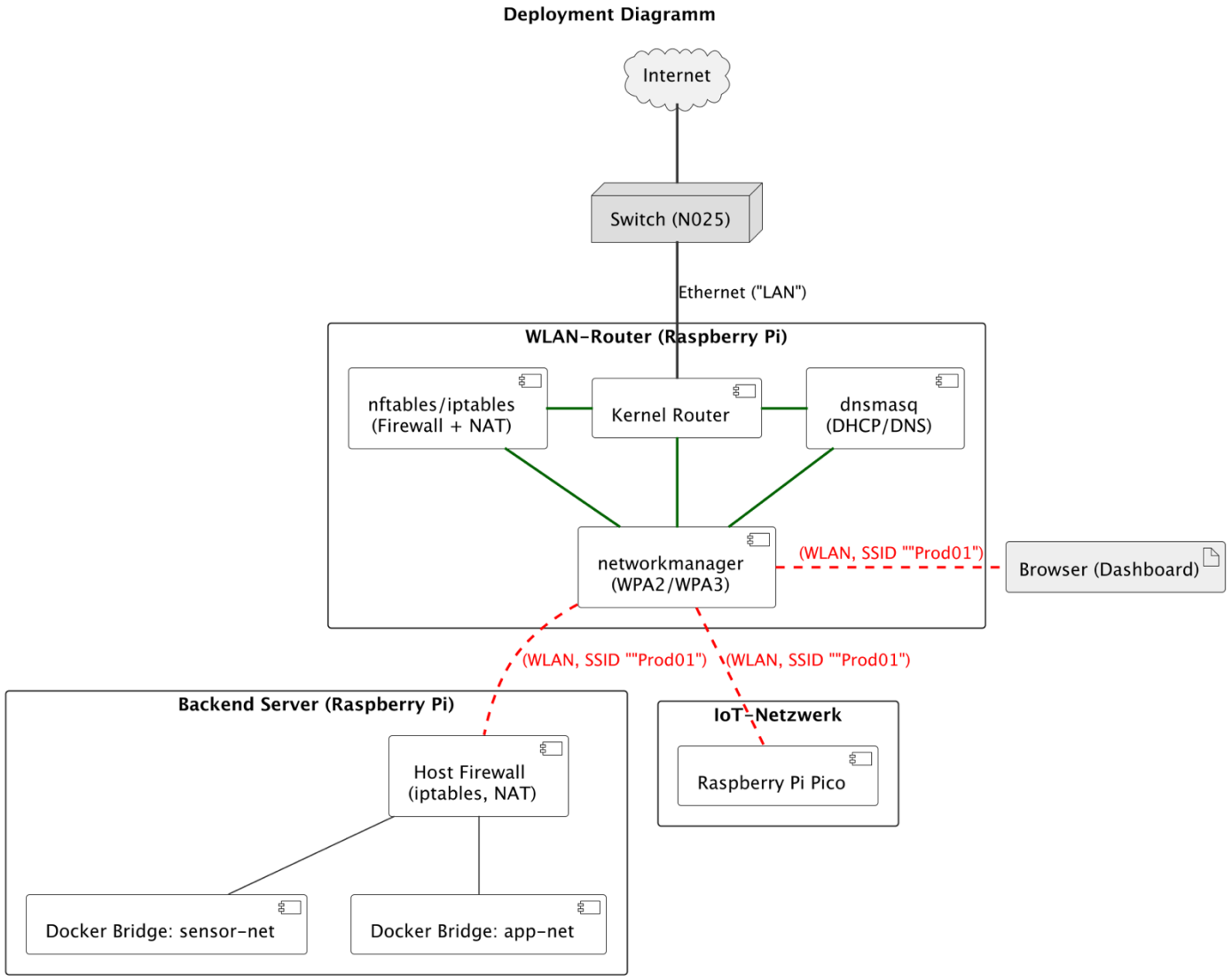
Anschlüsse: Hardware-Debugger



Raspberry Pi Pico: GPIO-Belegung



2.2 Architektur-Design



Entwicklung eines Architektur-Designs

Security-by-Design-Ansatz: Sicherheit wird von Anfang an in ein System/Software eingebaut. Sicherheit ist also ein Teil des Designs und nicht ein nachfolgend eingebauter Zusatz.

- Bei der Entwicklung eines Architekturdesigns sollte man von Beginn an Sicherheitsaspekte berücksichtigen.
  - Sicherheitsaspekte führen meist auch zu einer besseren Stabilität, Performance und Wartbarkeit des Gesamtsystems.
- Security-by-Design Anforderungen lassen sich in Form von Sicherheitsprinzipien formulieren.

Ein Sicherheitsprinzip ist eine anschauliche Daumenregel, die eine Denkrichtung beschreibt, die hilft ein sicheres System zu entwerfen.

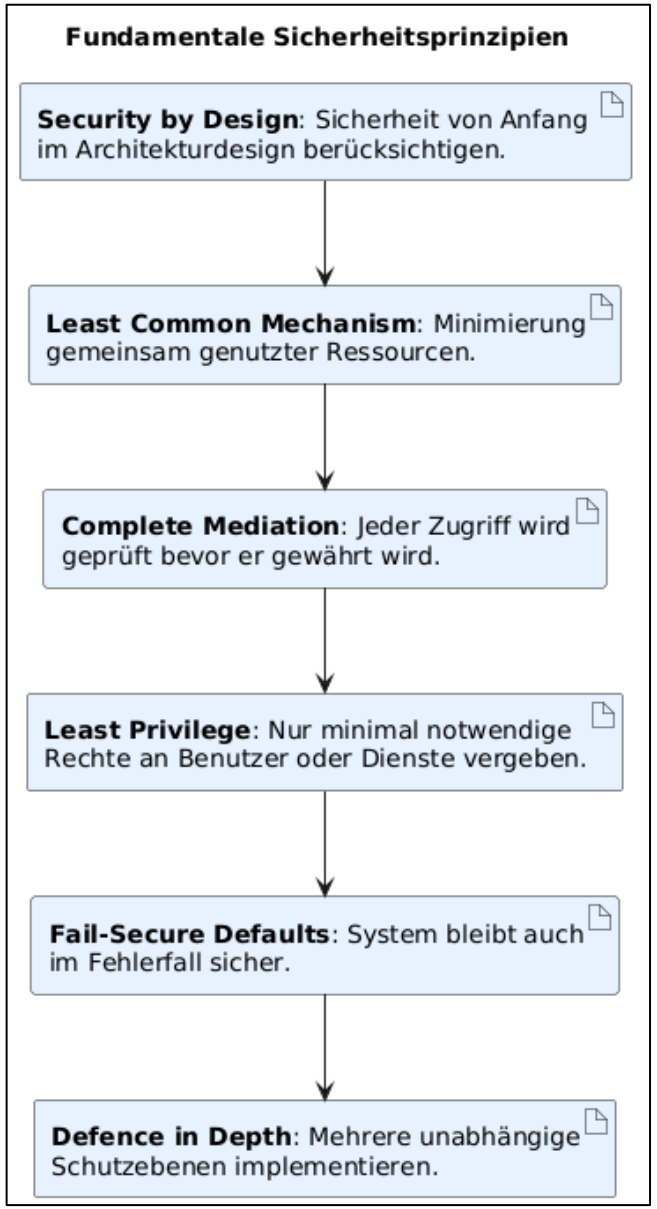
- Sicherheitsprinzipien sind
  - von jedem verständlich (Techniker und Anwender)
  - allgemein gültig
  - technologieunabhängig
- Sicherheitsaspekte führen meist auch zu einer besseren Stabilität, Performance und Wartbarkeit des Gesamt-systems.

Entwicklung eines Architektur-Designs

- Bei der Anwendung der Security Prinzipien geht man grundsätzlich von einem Zero Trust Ansatz aus.

> Zero Trust Ansatz: Es wird grundsätzlich keinem Nutzer oder System/Software vertraut, unabhängig davon, ob sie sich innerhalb oder außerhalb des LAN / Docker Netzwerkes / WLAN befinden.

- Die nebenstehende Abbildung zeigt eine Auswahl der wichtigsten Sicherheitsprinzipien.



Entwicklung eines Architektur-Designs

- **Least Common Mechanism**

  Die gemeinsame Nutzung von IT-Ressourcen muss minimiert werden, um unautorisierte Zugriffe und Sicherheitsverletzungen zu verhindern.

- Je mehr Systeme sich **denselben Mechanismus teilen**, desto größer ist die **Angriffsfläche.**
- Typische gemeinsam genutzte Ressourcen:
  - Hardware
  - Betriebssystem
  - Netzwerk
  - Kommunikationskanälen (Session, mqtt-Topic)
  - Benutzer und Passwort für einen Zugriff

- Sicherer Architekturansatz: Isolation der IT-Ressourcen
- **Beispiele:**
- Segmentierung von Netzwerken: Pro Netzwerk ein Applikationstyp.
  - Netzwerk für Produktion
  - Netzwerk für IoT (Internet of Things)
  - Netzwerk für Backend-Service
- Segmentierung von Applikationen/Prozessen auf einem Host
  - Pro Prozess ein eigener Docker-Container
  - Docker-Container je nach Anwendungskontext in unterschiedliche Docker-Netzwerke auftrennen
- Segmentierung von Kommunikationskanälen
  - Pro Endgerät ein eigener sichere TLS-Kanal
  - Pro Sensor, Aktor eine eigene Nachrichtenqueue

Entwicklung eines Architektur-Designs

- Segmentierung von Zugangsdaten (credentials) pro Endgerät oder pro zugreifender Person
  - Separater Benutzername und Passwort
  - Separate Zugriffsberechtigungen

- Anwendung für das Projekt:
  - Trennung des IoT-Netzwerkes und Backend-Netzwerkes vom "Unternehmens-LAN".
  - Verwendung von Docker und Docker-Netzwerke auf dem Backend-Server.
  - Firewall auf WLAN-AP und Backend-Server.
  - Verschlüsselte TLS-Verbindungen für jede bidirektionale Kommunikation.
  - Pro Sensor einen separaten Benutzer & Passwort.
  - Pro Sensor einen separate Nachrichten-Queue (Topic)
  - Jeder Sensor darf nur auf seine Nachrichten-Queue zugreifen (ACL)

Entwicklung eines Architektur-Designs

- **Complete Mediation**

  Jeder Zugriff auf eine IT-Ressource – sei es eine Datei, ein Prozess, ein Service oder ein Netzwerkobjekt – muss jedes Mal überprüft werden muss, bevor er gewährt wird.

- Kein Caching von Berechtigungen: Zugriffsrechte werden nicht gespeichert, sondern bei jedem Zugriff neu geprüft.
- Ständige Überprüfung: Selbst wenn eine Sitzung bereits authentifiziert ist, wird jeder einzelne Zugriff auf Dateien, Daten oder Ports neu autorisiert.
- Reaktion auf Berechtigungsänderungen: Wenn sich die Rechte eines Nutzers (z.B. von Admin zu Standard) mitten in einer Sitzung ändern, greift dies durch die ständige Prüfung sofort.

- Anwendung für das Projekt:
  - Docker-Netzwerk, Netzwerk: Firewall überprüft jede Nachricht gegen ihr internes Regelwerk, nur erlaubte Nachrichten werden durchgelassen.
  - Sicherer TLS-Kanal: TLS überprüft jede Nachricht (Integritätsprüfung, Verschlüsselung) ob Nachricht vom richtigen TLS-Kommunikationspartner kommt.
  - MQTT-Session mit TLS: Initiale Authentifizierung mit ClientID, Username, Password und Aufbau einer mit TLS verschlüsselten TCP-Session mit Sequence-Number als Authentifizierungsnachweis. MQTT-Broker überprüft bei jeder Anfrage die Berechtigung.
  - HTTP-Zugriff: Initiale Authentifizierung eines Benutzers und ausstellen eines Cookies oder JWT-Tokens, das bei jeder Anfrage übermittelt wird. Webserver bestimmt anhand des Cookies/Tokens die Berechtigung des Benutzers.

Entwicklung eines Architekturdesigns

- **Least Privilege (Prinzip der minimalen Rechte)**

  Benutzer, Systeme oder Anwendungen erhalten nur die minimalen Zugriffsrechte, die zur Ausführung ihrer spezifischen Aufgaben erforderlich sind.

- Verringerte Angriffsfläche: Durch die Einschränkung von Berechtigungen haben Angreifer weniger Möglichkeiten, Schaden anzurichten, falls ein Benutzerkonto kompromittiert wird.
- Benutzerrollen: Mitarbeiter sollten für alltägliche Aufgaben Benutzerkonten mit minimalen Berechtigungen (LPUA) verwenden, um das Risiko der Ausführung von Schadsoftware zu minimieren.
- Erhöhte Stabilität: Verringert versehentliche Schäden, die dadurch entstehen, dass Benutzer Systeme/Daten löschen oder verändern, auf die sie keinen Zugriff haben sollten.

- Rechte für Prozesse
  - eigener Benutzer pro Dienst, ohne root-Rechte
  - nur auf die Dateien zugreifen dürfen, die gebraucht werden
  - nur die Netzwerkports nutzen dürfen, die gebraucht werden
  - keine Admin-Rechte auf andere Dienste haben
- Zugriffsrechte auf Netzwerke
  - Firewall erlaubt nur die Netzwerkzugriffe von Hosts zulassen, die für die Applikation benötigt werden
  - Docker Container in getrennten Docker Netzwerken können nicht aufeinander zugreifen.
- Webserver – DB-Zugriff
  - Webserver erhält pro Aufgabe einen eigenen DB-User mit minimalen Rechten (kein DB admin):
    SELECT, INSERT, UPDATE auf Tabelle sensor_data

Anwendung auf das Projekt

- **Least Common Mechanism**
- A. Physische Segmentierung
  - IOT-Geräte und Backend-Services sind vom Unternehmens-LAN isoliert.
  - IoT-Geräte vom Klebestand werden über einen eigenen Mikrocontroller gesteuert.
  - Mikrocontroller nur über das separierte Produktions-WLAN verbunden.
  - Der zugehörige WLAN-AP wird auf separaten Raspberry Pi installiert und stellt nur das Produktions-WLAN zur Verfügung.
- B. Logische Segmentierung im Backend-Host
  - Docker-Bridge stellt verschiedene Docker-Netzwerke für die einzelnen Applikationstypen bereit (Kommunikation mit IoT-Geräten, Backend-Services zur Speicherung der Daten).

    → Datenbankserver nur via Reverse-Proxy erreichbar
  - Jeder Backend-Service läuft isoliert in seinem eigenen Container.
- C. Logische Segmentierung der Datenbankzugriffe via Web Server
  - Einen Benutzer "iot_write_user" und DB-Pool und für den Schreibzugriff auf die Datenbank
  - Einen Benutzer "iot_read_user" und DB-Pool Zugriff für den Lesezugriff via Dashboard.

Anwendung auf das Projekt

- **Complete Mediation**
  - Netzzugangskontrolle: Zugang zum Produktions-WLAN nur nach Authentifizierung möglich → Authentifizierung und Autorisierung bei jeder Neuverbindung nötig.
  - Verschlüsselte und Integritätsgesicherte WLAN-Verbindung (WPA2/WPA3), die bei jedem Zugriff den Client authentifiziert und autorisiert.
  - nftables/iptables – Firewall auf WLAN-AP
    Paketfilterung mit Whitelisting.
    NAT (Network Adress Translation) für Zugriff in das Unternehmens-WLAN
    → nur IP-Pakete vom Production-Netzwerk in das Unternehmens-LAN sind erlaubt.
  - Docker überprüft jeden Zugriff (iptables-FW) auf Berechtigung.
  - Reverse Proxy mit Web Application Firewall (WAF).
  - Sichere Kommunikationskanäle via TLS und Zertifikaten für jede Form der Netzwerkkommunikation
    → jede Nachricht wird auf Authentizität und Integrität geprüft.
    → Gesicherte Kommunikation zwischen IoT-Geräten und Backend via TLS
    → Gesicherte Kommunikation zwischen Services im Backend via TLS
    → Gesicherte Kommunikation zwischen Browser und Backend via TLS
    → Ausstellung von Zertifikation via eigener CA
    → JWT-Tokens für Benutzeranfragen via Web-Dashboard

Anwendung auf das Projekt

- **Least Privilege**
  - Nur definierte Ports werden zwischen Containern und der Außenwelt freigegeben.
    → Docker überprüft jeden Zugriff
    → nur nginx und mqtt-Broker habe Zugriff nach "außen"
  - IoT-Benutzer auf Mikrocontroller – Sensor sensor01 erhält nur Schreibrechte für seine Queue "sensor01/data" auf dem Message-Broker
  - IoT-Benutzer auf Backend-Service controller.py erhält nur Leserechte auf die Queue "sensor01/data" und Schreibrechte auf die Queue "actuator01/data"
  - DB-Benutzer "iot_write_user" auf WebServer enthält nur Lese- und Schreibrechte auf die sensor_data-Tabelle in der Sensordatenbank sensor.
    (keine Rechte zum Anlegen von Datenbanken, Benutzern, Tabellen oder Rechte zur Rechtevergabe)
  - DB-Benutzer "iot_write_read" auf WebServer enthält nur Leserechte auf die sensor_data-Tabelle in der Sensordatenbank sensor.

2.3 Raspberry PI OS Installation

Raspberry PI Installation

- Mit Raspberry Pi Imager kann ein Raspberry Pi OS auf einer microSD-Karte installiert und sofort mit Ihrem Raspberry Pi verwendet werden.
- Laden Sie Raspberry Pi Imager herunter und installieren Sie ihn auf ihrem Entwicklungs-PC mit SD-Kartenleser.
  - Stecken Sie die microSD-Karte, die Sie mit Ihrem Raspberry Pi verwenden möchten, in den Kartenleser und starten Sie Raspberry Pi Imager.
    - 👉 Wählen Sie ihr Raspberry PI Modell aus
    - 👉 Wählen Sie das OS aus: Raspberry PI OS (64Bit)
    - 👉 Wählen Sie ihre SD-Karte aus
  - Wenn Sie dann auf weiter klicken, erhalten Sie die Möglichkeit verschiedene Konfigurationseinstellungen für ihr zu installierendes System vorab zu bestimmen.

Konfigurationseinstellungen im Raspberry Pi Imager

- Die im Raspberry PI Imager möglichen Konfigurations-einstellungen sind im folgenden gelistet.
- Die Optionen b) und c) sind für ein Headless-Setup (kein Bildschirm vorhanden) essentiell:
  - **Hostname:** Name des Raspberry Pi im Netzwerk
    - 👉 WLAN-Router : "WLAN-AP-Router"
    - 👉 Backend-Server : "Backend-Server" :
  - **Enable SSH:**
    - 👉 Ermöglicht eine Remote Anmeldung an einem entfernten Host und stellt eine Remote Konsole (Shell) zur Verfügung

```
$ ssh username@hostname
```

  - **Set locale settings**
    - 👉 Haupstadt, Zeitzone, Tastaturlayout
    - 👉 z. B.: Berlin, Europe/Berlin, de
  - **User & Passwort**
    - 👉 Benutzername (keine Standardnamen: admin, Admin, administrator, Administrator, ...)
    - 👉 Passwort (>8 Zeichen mit Zahl und Sonderzeichen) festlegen
  - **Fernzugriff einrichten**
    - 👉 Um die Fernwartung nutzen zu können, muss zuerst ein Benutzer auf der Webseite Raspberry Pi Connect anlegt werden, für den man zwingend die 2-Faktor-Authentifizierung aktivieren sollte.

Zusatzsoftware installieren

- apt ist ein Paketverwaltungs-Tool auf Linux-Systemen (vor allem Debian/Ubuntu), zur Installation, Aktualisierung und zum Entfernen von Software.
- Das apt-Tool greift auf öffentliche SW-Repositories

```
/etc/apt/sources.list
```

- Das apt-Tool arbeitet mit Root-Rechten (sudo) und löst automatisch Abhängigkeiten zu anderen SW-Paketen auf und installiert diese.
- sudo steht für „superuser do“ und ermöglicht einen nicht priviligierten Benutzer Befehle mit Administrator-Rechten ausführen (Least Privileges!).
  - Dazu muss der Benutzer-Account durch einen Administrator einmalig in die Gruppe "sudo" aufgenommen werden

```
$sudo usermod -aG sudo BENUTZERNAME
```

- Typische Befehle
  - Verfügbare Paketliste aktualisieren

```
$sudo apt update
```

  - Updates für installierte Pakete installieren

```
$sudo apt upgrade -y
```

  - Eine neue Software installieren

```
$sudo apt dnsmasq
```

  - Eine Software deinstallieren

```
$sudo apt remove nginx
```

Raspberry Pi Connect: Fernwartung für Rasbberry PI

- Raspberry Pi Connect bietet sicheren Remote-Zugriff (Fernwartungszugriff) auf einen Raspberry Pi über einen Webbrowser.
- Der Webbrowser baut eine sichere direkte Peer-to-Peer-Verbindung mit dem WebRTC-Protokoll zwischen dem Client-Browser und dem jeweiligen Raspberry-PI System auf.
- Installation:

```
$sudo apt update
$sudo apt upgrade -y
$sudo apt install rpi-connect
$rpi-connect on
$rpi-connect signin
Complete sign in by visiting
https://connect.raspberrypi.com/verify/XXXX-XXXX
```

- Um die Fernwartung nutzen zu können, muss zuerst ein Benutzer auf Raspberry Pi Connect anlegt werden, für den man zwingend die 2-Faktor-Authentifizierung aktivieren sollte.
- Die Verbindung über WebRTC ist verschlüsselt und integritätsgesichert (Complete Mediation).

Folgende interessante Webseiten:

- Dokumentation: https://www.raspberrypi.com/documentation/services/connect.html
- Seite zum Verbinden mit den registrierten Geräten https://connect.raspberrypi.com/devices

Rasberry PI Grundkonfiguration

- Rasberry PI Configuration:
  - Mittels dem Tool raspi-config lassen sich verschieden Einstellungen auf dem Host vornehmen

$ sudo raspi-config

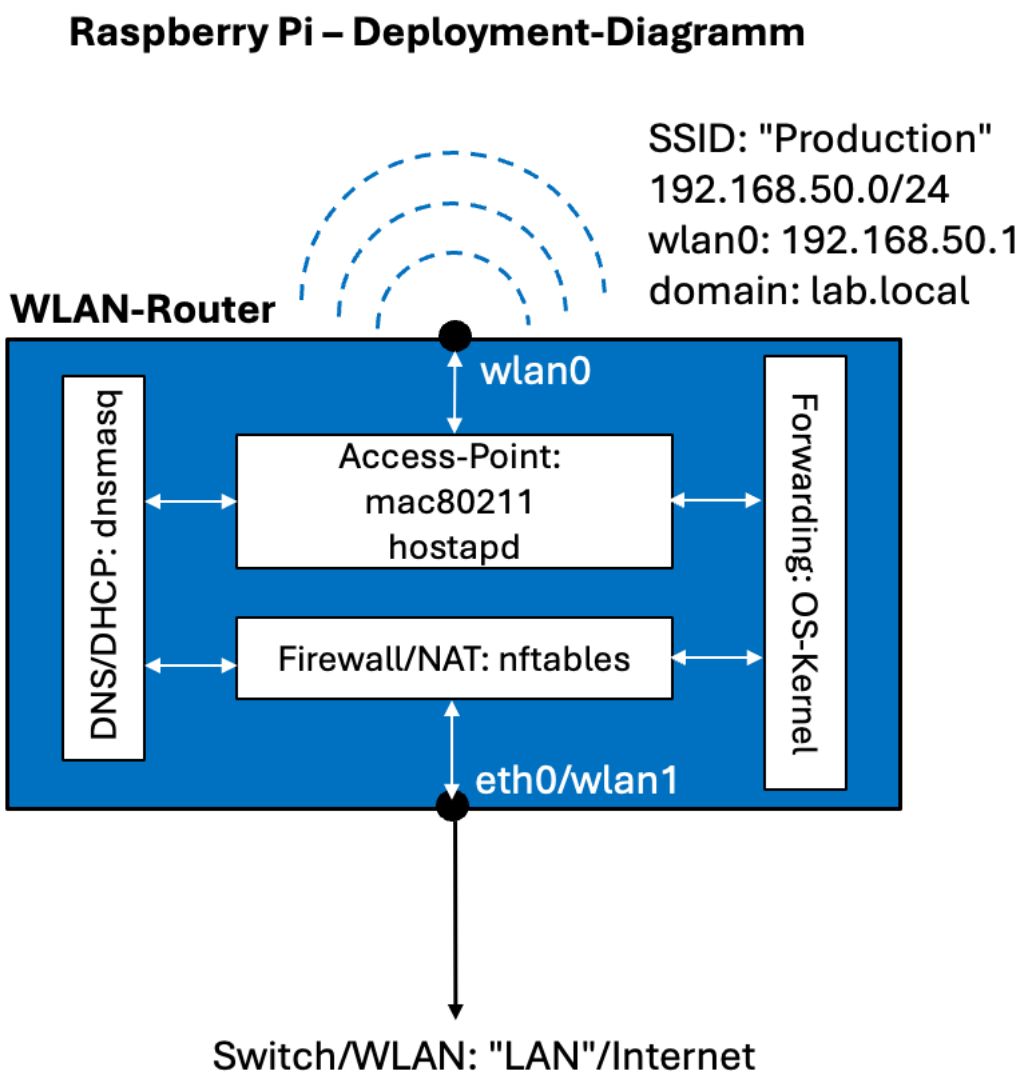
```
Locale: German / Germany / UTF-8
Time Zone: Europe/Berlin
Keyboard: Apple / German / German
WLAN-Country: DE / Germany
```

- Die gleichen Einstellungen können auch über die grafische Oberfläche vorgenommen werden:

  Menu → Preferences
- Dafür braucht man ein lokales physisches Display **oder** Remote-Zugriff per Raspberry PI Connect (Screen Sharing)

- Tastaturkombinationen für MAC OS und Remote Connect
  - option + ? = "\"
  - option + ° = "|"

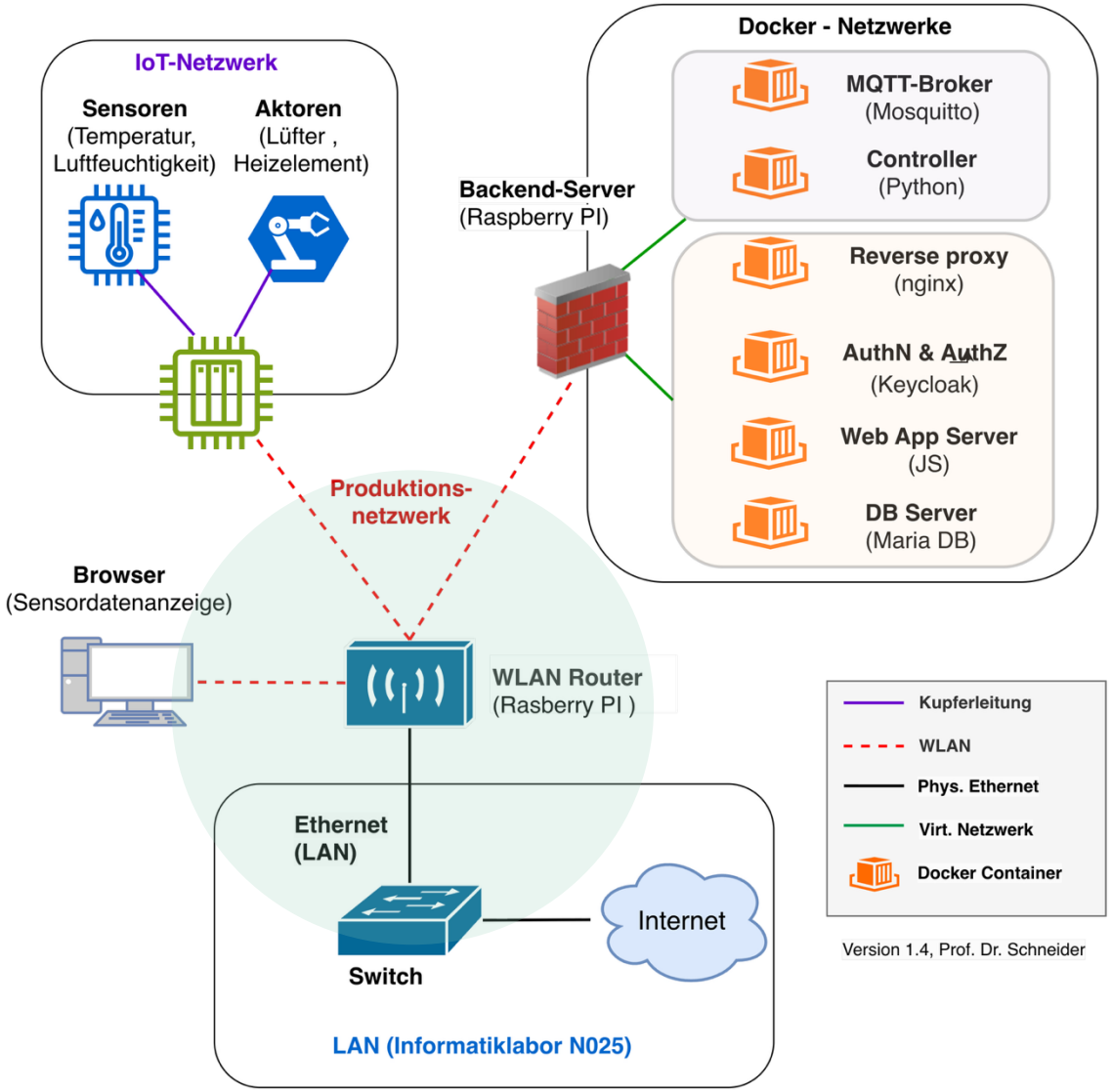
2.4 WLAN-AP und Router



WLAN-Router

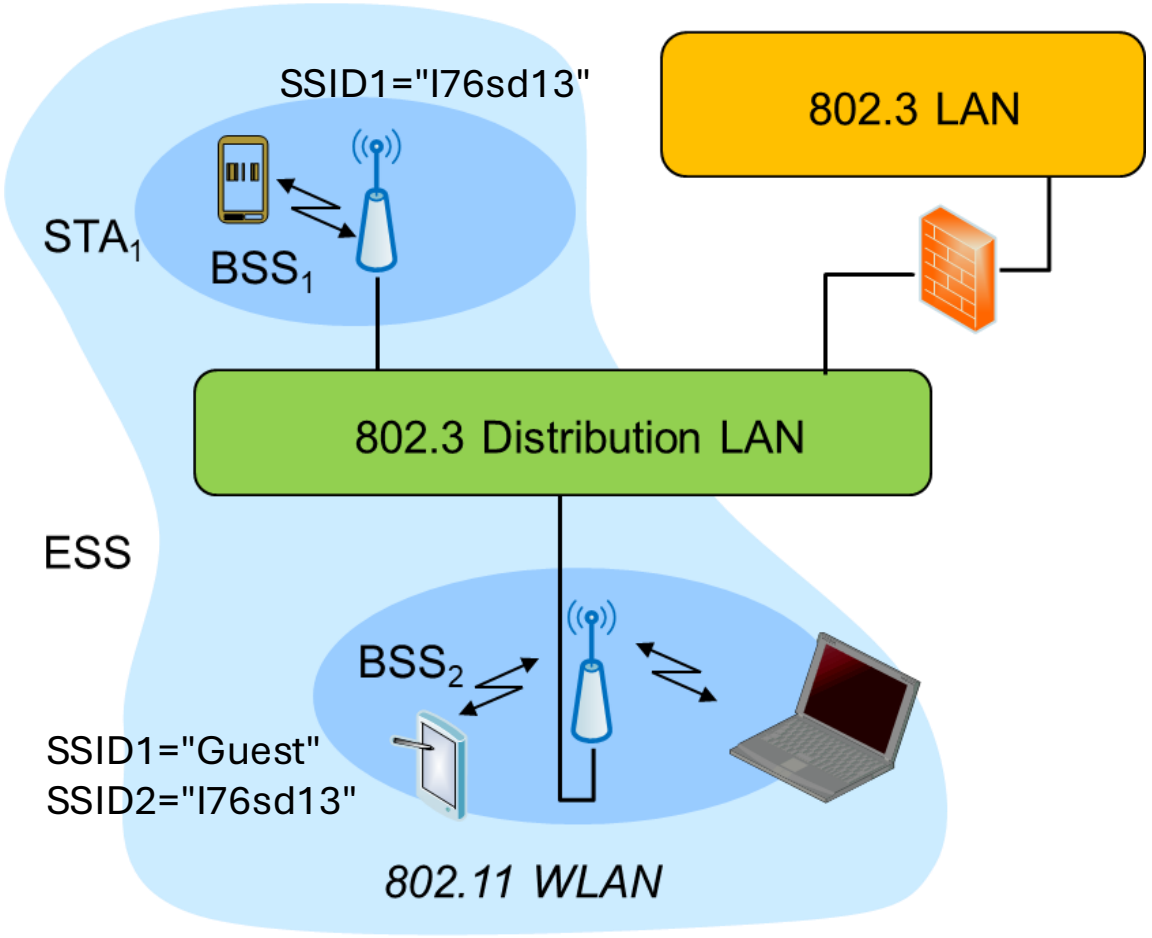
- Zur Installation eines WLAN-Router steht Ihnen ein Raspberry PI-Minicomputer zur Verfügung.
- Der WLAN-AP stellt das WLAN-Netzwerk "Production" bereit.
- WLAN-Router: Weiterhin routet er von einer internen Firewall erlaubte Datenpakete in das LAN des Informatiklabors um beispiels-weise
  - SW-Pakete installieren zu können,
  - das Remote Management zu ermöglichen.
- Bevor mit der Installation von benötigter Software begonnen wird, sollte das System auf den aktuellen SW-Stand gebracht:

```
$sudo apt update && sudo apt upgrade -y
```



WLAN (IEEE802.11)

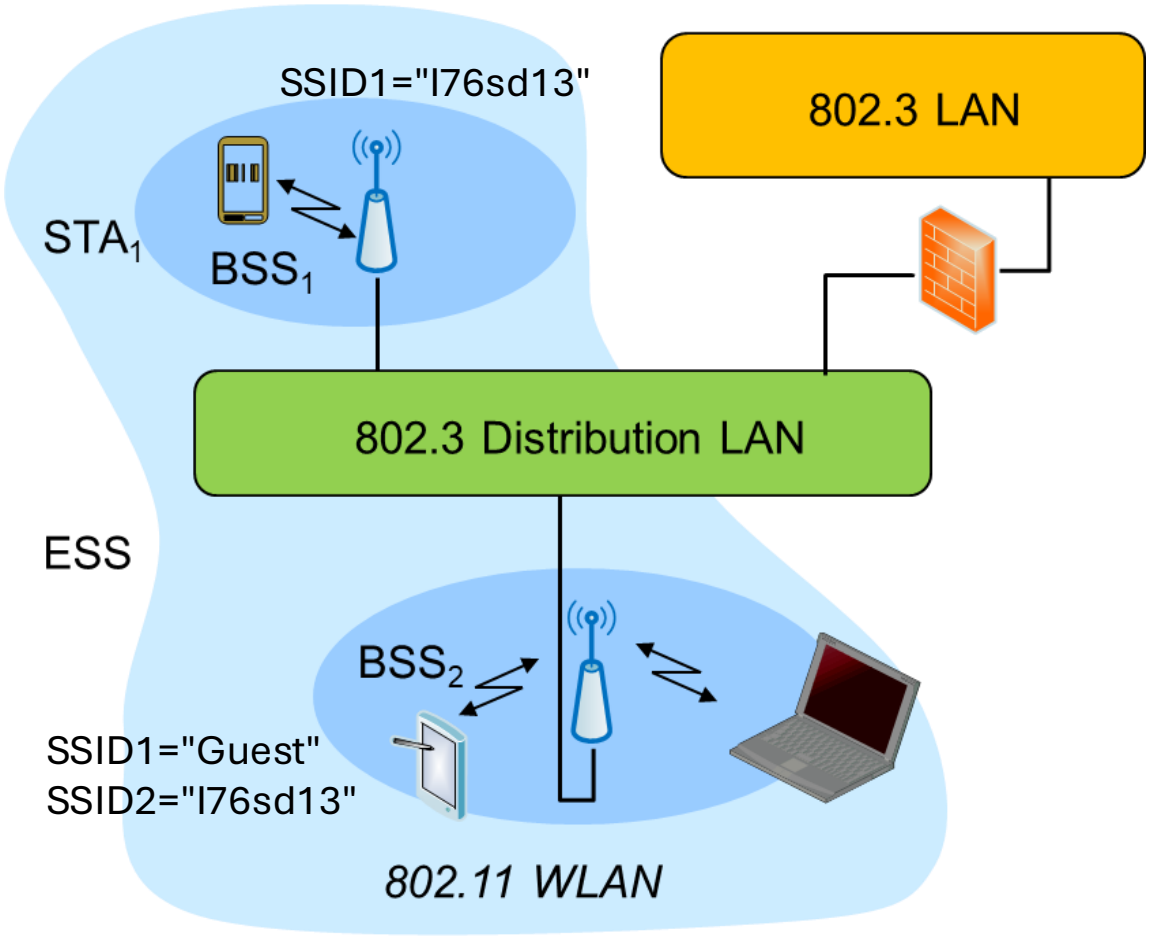
- WLAN (Wireless LAN): Netzwerkprotokoll für die flexible und mobile Anbindung von tragbaren Endgeräten in Firmen, Hochschulen, Heimnetzwerken, Hotspot, ...
- Markenname: WiFi (Wireless Fidelity) in Analogie zu HiFi (High Fidelity: höchste Wiedergabetreue))
- Ein WLAN-Netzwerk wird über einen Namen seine SSID (Service Set Identifier) gekennzeichnet und verwendet einen oder mehrere WLAN-Access-Points zum Aufbau des Netzwerkes.
- Ein Access-Point kann mehrere SSIDs bereitstellen.
- Durch den Einsatz von mehreren Access-Points kann ein Firmen Campus räumlich in mehrere Funkzellen eingeteilt werden. Diese verteilten Funkzellen werden als Extended Service Set (ESS) bezeichnet.
- Eine Funkzelle wird auch als Basic Service Set (BSS) bezeichnet.



Begriff	Bedeutung
SSID	Name eines WLAN-Netzes.
BSS (Basic Service Set)	Ein Access Point.
ESS (Extended Service Set)	Gruppe mehrerer Access Points mit gleicher SSID.

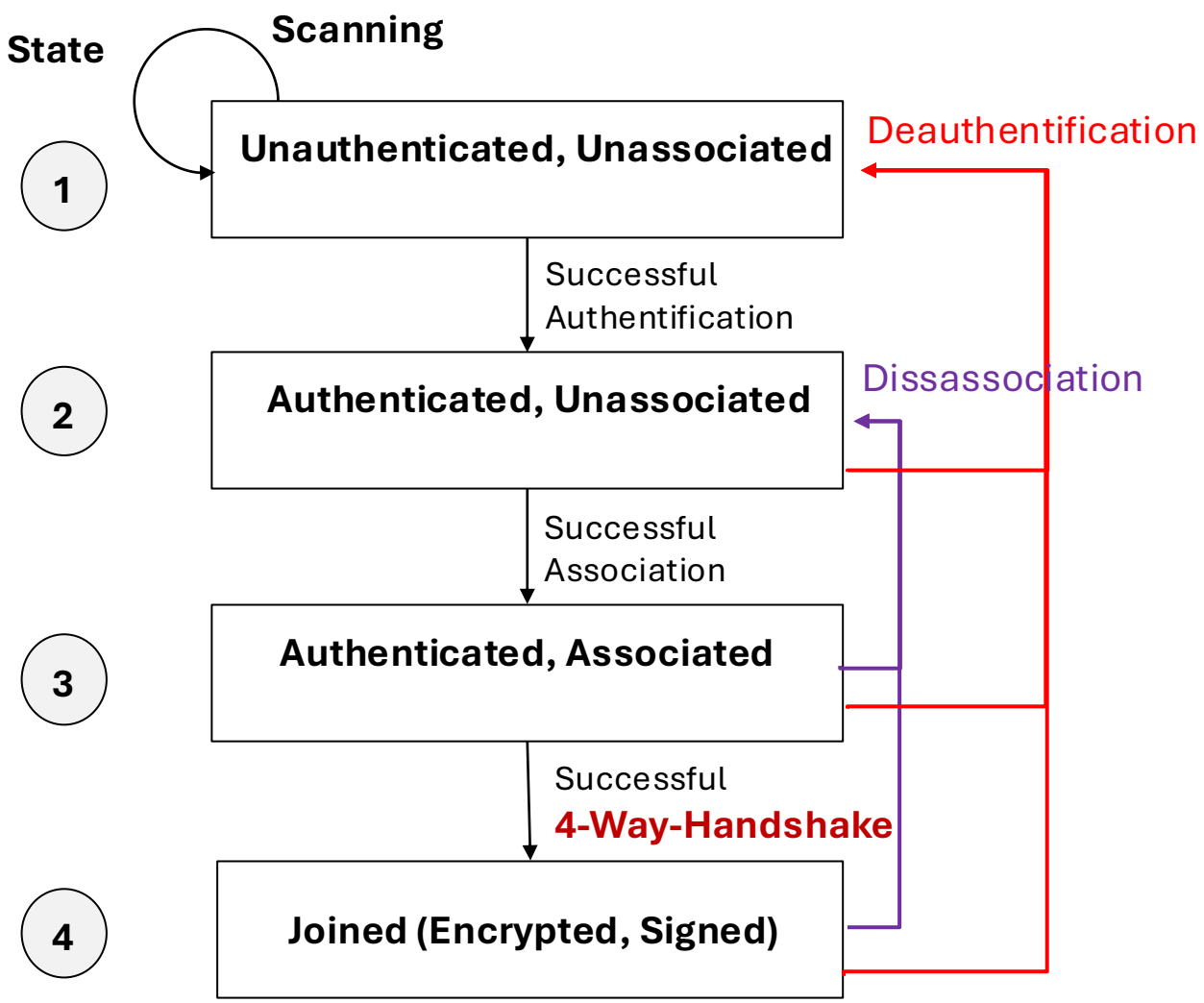
WLAN (IEEE802.11)

- Die WLAN-Clients kommunizieren über diese Access-Points untereinander
  - Innerhalb einer Funkzelle (BSS) erfolgt die Kommunikation über den AP der jeweiligen Funkzelle.
  - Zwischen mehreren Funkzellen erfolgt die Kommunikation über ein kabelgebundenes Distribution System und den jeweiligen APs.
  - WLAN-Clients können über das Distribution-LAN in Abhängigkeit der Firewall Konfiguration auch mit weiteren Hosts im LAN und im Internet kommunizieren.



WLAN-Connection-Zustandsmaschine

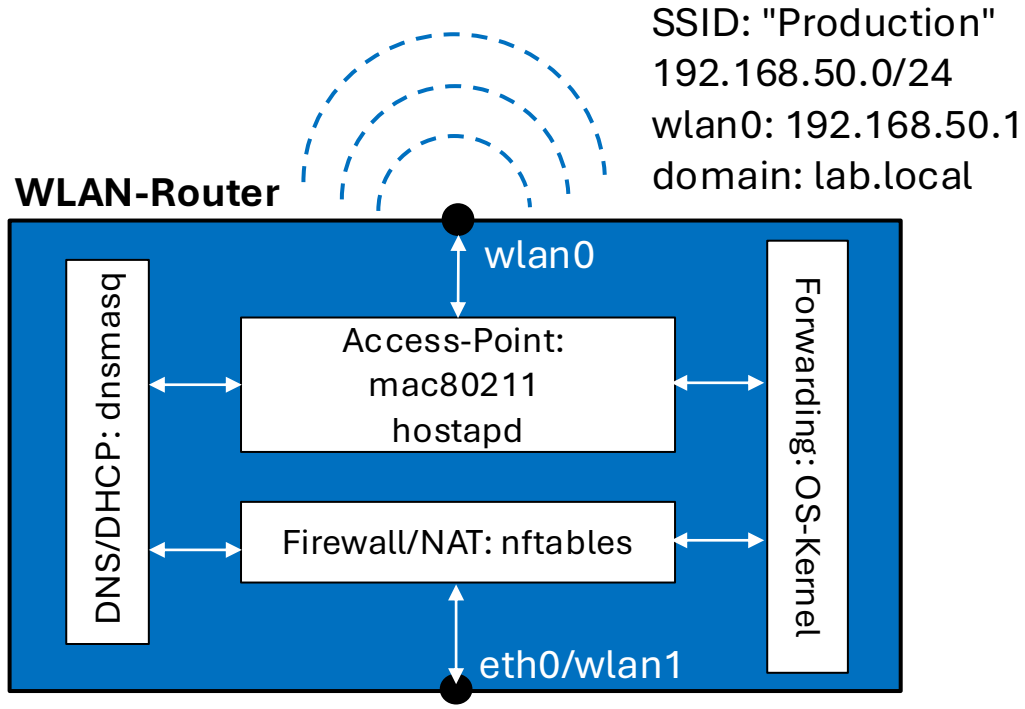
Aktion	Beschreibung
Scanning	Der Client sucht nach verfügbaren WLANs (aktive oder passive Suche).
Authentication	Der Client und AP führen eine Authentifizierung durch (z. B. Open System oder SAE).
Association	Der Client bittet um Assoziation, AP prüft ob er noch freie Ressourcen besitzt und der Client seinen geforderten WLAN-Standard (802.11n, 802.11ac, 802.1ax unterstützt.
4-Way Handshake	WPA2/WPA3-Negotiation: Aushandlung der Sitzungs-schlüssel und gegenseitige Authentifizierung zwischen AP und STA.
Joined	Verbindung erfolgreich. Client erhält die IP-Konfiguration über DHCP. Daten können verschlüsselt und signiert ausgetauscht werden.



Raspberry Pi als WLAN-Router einrichten (WLAN-Access-Point)

- Das Raspberry PI OS stellt zur Einrichtung, Überwachung und fortlaufenden Verwaltung den Dienst **Network Manager** zur Verfügung.
- Mit dem Network Manager kann ein WLAN Access Point (AP) konfiguriert wird.
  - Der AP verwendet eine LINUX Kernelfunktionalitäten namens mac80211 (IEEE 802.11 WLAN-Protokoll) zur Bereitstellung der Funkzelle.
  - Die Authentifizierung und Autorisierung von Teilnehmer am Funknetzwerk wird über den Daemon hostapd durchgeführt (WPA2- oder WPA3-Standard).
  - Als Interface für den WLAN-AP wird wlan0 verwendet.
  - SSID: "Production"
- Der Network Manager wie folgt gestartet werden
  - $sudo nmcli : Network Manager Command Line Interface
  - $sudo nmtui : Network Manager Text User Interface

Raspberry Pi – Deployment-Diagramm




```
# Daemon hostapd für die Netzzugangskontrolle
  installieren
  $sudo apt install hostapd -y
```

Konfiguration des AP

- Der Netwerkmanager konfiguriert die Netzwerk-Schnitt-stellen für den WLAN-AP und aktiviert diese beim System-Neustart.
- Das Tool nmcli greift auf den Networkmanager zu und ermöglicht das Anlegen eines WLAN-APs.
- Beispiel: Für das Anlegen eines WLAN-AP ist die Angabe eines Verbindungsnamen `ap-wlan0`, die Angabe eines WLAN-Adapter `wlan0` und die `SSID "Production"` der Funkzelle nötig:

```
# Verbindung ap-wlan0 für wlan0 anlegen
# (falls noch keine existiert)
$sudo nmcli connection add type wifi ifname wlan0
      con-name ap-wlan0 mode ap ssid Production
```

- Die Konfiguration wird in der einer Datei, mit dem gewählten Verbindungsnamen ap-wlan0 gespeichert und kann dort auch manuell ergänzt oder angepasst werden.

```
$sudo nano /etc/NetworkManager/system-
connections/ap-wlan0.nmconnection
```

- Der Status aller Netzwerk-Interfaces eines Hosts kann ebenfalls mittels nmcli angezeigt werden

```
$sudo nmcli device status
```

Konfiguration AP

- Möchte Sie ihren erzeugten AP anschliessend modifizieren, können Sie sich alle aktiven Profilnamen anzeigen lassen:

```
$sudo nmcli connection show

NAME            UUID                TYPE    DEVICE
preconfigured   5c06cb3c-9[...]     wifi    wlan1
ap-wlan0        1913d211-a[...]     wifi    wlan0
```

- Anschliessend können Sie beispielsweise ihr WLAN-Passwort (> 12 Zeichen) verändern

```
$sudo nmcli connection modify ap-wlan0
            wifi-sec.psk "Production-01"
```

- Abschließend den AP neu starten

```
$sudo nmcli connection down ap-wlan0
$sudo nmcli connection up ap-wlan0
```

Konfiguration des AP

- Das Programm **iw** ist ein Kommandozeilen-Tool unter Linux zur Konfiguration und Verwaltung von WLAN-Adaptern.
  - Beispiele: Funkzellen-Scan, Kanalwahl, Betriebsmodus, Sendeleistung, ...
- **iw** nutzt die Schnittstelle cfg80211, die Teil des Linux-WLAN-Kernel-Stacks ist, und kommuniziert mit der WLAN-Schnittstelle mac80211.
- Mittels des folgenden Befehles kann der Status und die mit dem AP verbundenen Stationen angezeigt werden:

```
$sudo iw dev wlan0 station dump
```

- Der folgende Befehl zeigt verfügbare WLAN-Netzwerke (SSID, Kanal, Signalstärken, Verschlüsselung

```
$sudo iw dev wlan0 scan
```

- Setzen des Betriebsmodus
  - Auf wlan0 einen WLAN-AP (Funkzelle ohne WPA) betreiben

```
$sudo iw dev wlan0 set type __ap
```

  - Um WLAN-Pakete anderer Funknetze/Stationen mitlesen zu können, kann der Adapter in den Monitor-Mode versetzt werden

```
$sudo iw dev wlan0 set type monitor
```

DHCP/DNS-Server einrichten

dnsmasq – Leichtgewichtiger DNS-Forwarder und DHCP-Server

- Um einen funktionsfähigen WLAN-AP zu betreiben, benötigt man 2 Kernfunktionalitäten:
- DHCP (Dynamic Host Configuration Protocol):
  - Vergibt automatische IP-Adressen an WLAN-Clients.
  - Sobald sich ein Gerät mit dem WLAN verbindet ("Join), bekommt es von dnsmasq eine IP-Adresse aus einem vorab konfigurierten Subnetz (z. B. 192.168.50.0/24)
- DNS (Domain Name System):
  - Beantwortet die DNS-Anfragen von WLAN-Clients, also liefert die Umwandlung von DNS-Namen (z.B.: www.google.com) in IP-Adressen.
  - Dazu forwarded er die Anfrage an einen Upstream-DNS-Server (z. B. 8.8.8.8. – Google DNS, 1.1.1.1 Cloudflare DNS) und cached die Antwort für Folgeanfragen lokal.

```
# Paketliste aktualisieren und dnsmasq installieren
  $sudo apt update
  $sudo apt install dnsmasq -y
```

Starten und Stoppen von Diensten am Beispiel von dnsmaq

- **systemd** ist das Init-System moderner Linux-Systeme.
- Es startet, stoppt und verwaltet Systemdienste, steuert deren Startreihenfolge und überwacht deren Ausführung während des Systembetriebs.

```
$sudo systemctl start/stop/restart dnsmasq
```

- Mittels systemd kann der Dienststatus von dnsmaq überprüft wwerden

```
$sudo systemctl status dnsmasq
```

- systemd enthält auch ein Log-System namens journald. Die Anzeige der Logs von dnsmasq erfolgt mittels

```
$sudo journalctl -u dnsmasq
```

- **Fehleranalyse:**
  - Anzeige des Status des Services ggf. mit letzter Fehlermeldung

```
$ sudo systemctl status -l dnsmasq
```

  - Anzeige der letzten Log-Einträge (`-e end of log`) des dnsmasq Dienstes

```
$sudo journalctl -e -u dnsmasq.service
```

  - Zeigt neue Logs in Echtzeit (`-f follow`)

```
$sudo journalctl -f -u dnsmasq.service
```

Starten und Stoppen von Diensten am Beispiel von dnsmaq

- Mit dem folgenden Befehl wird der Dienst dnsmasq beim Systemstart automatisch gestartet:

```
$sudo systemctl enable dnsmasq
```

- Technisch erstellt dann systemd einen symbolischen Link im Verzeichnis,

```
/etc/systemd/system/multi-user.target.wants/dnsmasq.service
```

auf die Unit-Datei des jeweiligen Dienstes, die den Dienststart definiert:

`/lib/systemd/system/dnsmasq.service`

- Bei der Konfiguration von dnsmasq (siehe hinten) wird das Interface wlan0 angegeben, über das dnsmasq arbeiten soll. dnsmasp versucht beim Start dieses Interface zu binden. Wenn das nicht gelingt arbeitet der Dienst nicht.

- Damit dnsmasq erst startet, nachdem das Interface wlan0 aktiv ist, muss der DNS/DHCP-Dienst (z. B. dnsmasq) in seiner Unit-Datei eine Abhängigkeit zum Netzwerk erhalten.

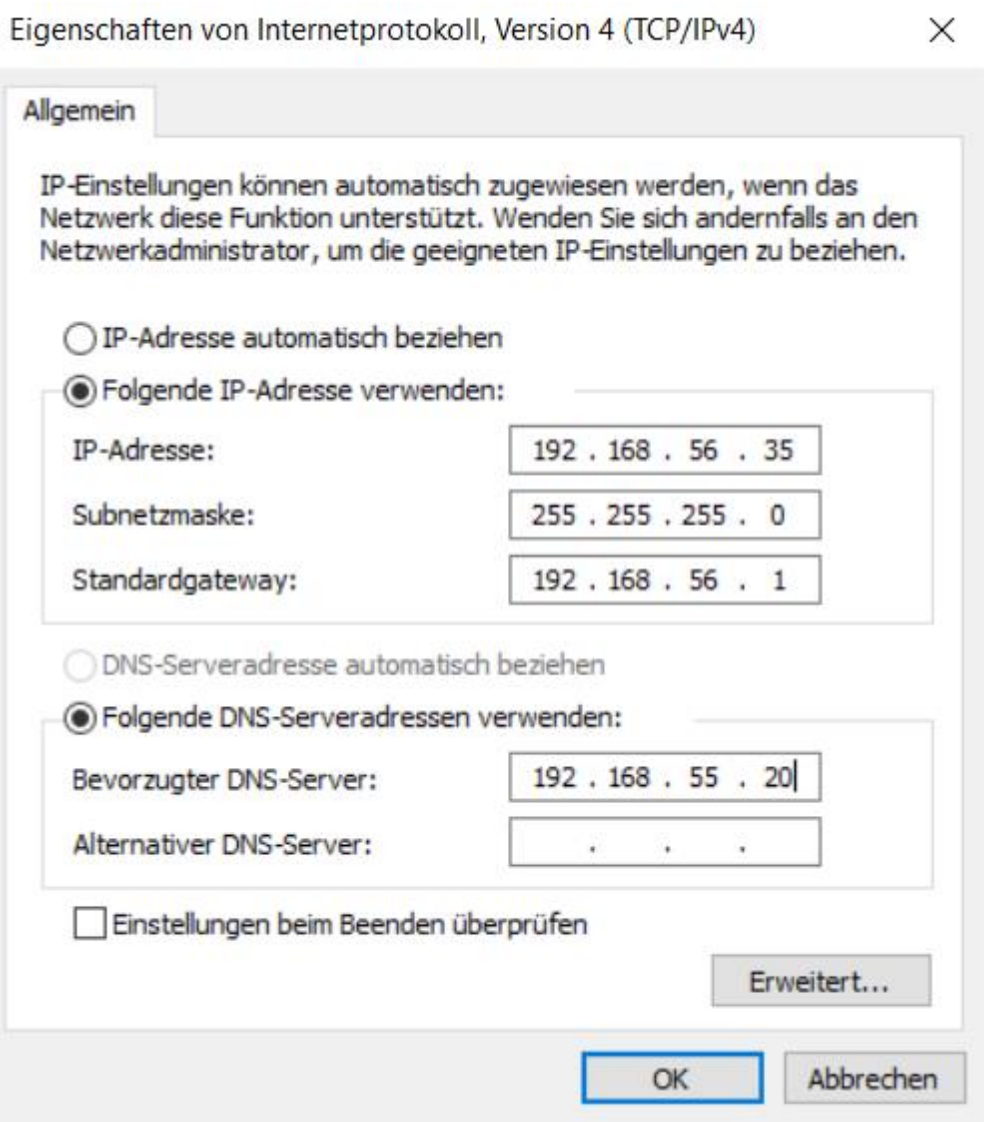
```
$sudo systemctl edit dnsmasq.service

[Unit]
Description=Load dhcp /dns after network
After=network-online.target
Wants=network-online.target
```

After=	Dienst startet erst **nach** dem das Netzwerk vollständig aktiv ist.
Wants=	Netzwerk soll ebenfalls gestartet werden.

IP-Adressvergabe und DHCP

Wie bekommt ein Host seine IPv4-Adresse?

- **Alternative 1**: Durch manuelle Konfiguration des Netzwerkadapters und pflegen der Daten in einer Excel-Liste!
  - IPv4-Adresse
  - Subnetzmaske
  - Standard Gateway (Default Router)
  - DNS-Server (lokaler DNS-Server)
- **Alternative 2:** (Dynamic Host Configuration Protocol) Dynamisches Beziehen der IP-Konfiguration von einem DHCP (Dynamic Host Configuration Protocol) Server.
  - Host besitzt initial keine IPv4-Adresse.
  - Fordert beim Neustart die IP-Konfiguration automatische an.
  - Erhaltene IP-Adressen besitzen eine Gültigkeitsdauer (z.B.: 24h) oder können dauerhaft an einen Host vergeben werden.



Konfiguration DHCP

- Die Konfiguration des WLAN-Routers als DNS- und DHCP-Server für das "Production" Netzwerk erfolgt in der Datei `/etc/dnsmasq.conf`.
- **Vorgehen:**
  - Backup der Standardkonfiguration

```
$sudo mv /etc/dnsmasq.conf /etc/dnsmasq.conf.backup
```

  - Editieren der Datei und Eingabe der gewünschten Konfiguration

```
$sudo nano /etc/dnsmasq.conf
```

  - Wichtig: Der WLAN-AP ist Default-Gateway und DHCP-Server für die Clients im Netzwerk "Production".

Ausschnitt aus der dnsmasq.conf

```
# Interface auf dem dnsmasq läuft
interface=wlan0
bind-interfaces    #bind dnsmasq only to wlan0
# Domain für lokale Geräte
domain=lab.local   #dns-domäne
local=/lab.local/  #Namen werden von dnsmasq beantwortet
# IP-Adressbereich mit 24h Lease-Time
dhcp-range=192.168.50.20,192.168.50.150,255.255.255.0,24h
```

Konfiguration DHCP auf dnsmasq

- Statische IP-Adressen und statische DNS-Einträge lassen sich in der dsnmasq.conf mit dem Schlüsselwort dhcp-host definieren.
- Beispiel: Vergabe einer festen IP-Adresse 192.168.50.20 und eines Hosts-Namens mcu für den Mikrokontroller

```
dhcp-host=AA:BB:CC:DD:EE:FF,mcu,192.168.50.21
```

MAC-Adresse IP

Hostname

- Der vollständige DNS-Name unter dem der Mirocontroller erreichbar ist lautet dann

Domain

mcu.lab.local

DHCP-Funktionstest

- **Test1 (DHCP):** Verbinden eines Clients mit dem WLAN-AP und Anzeige der Konfiguration des Clients-WLAN-Adapters mittels

```
Client:$ ifconfig wlan0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>
       mtu 1500
       inet 192.168.50.55  netmask 255.255.255.0
       broadcast 192.168.50.255
       …
```

- **Test2 (DNS):** Ping auf einen DNS-Namen im Internet (www.google.com) , um Namensauflösung zu testen.

```
Client:$ ping www.google.com
PING www.google.com (142.251.36.228) 56(84) bytes of data.
64 bytes from muc11s22-in-f4.1e100.net (142.251.36.228):
icmp_seq=1 ttl=119 time=17.9 ms
64 bytes from muc11s22-in-f4.1e100.net (142.251.36.228):
icmp_seq=2 ttl=119 time=17.6 ms
```

Aktivieren des Routings

- Routing: Damit der Datenverkehr vom wlan0-Interface an ein weiteres Host-Interfaces geforwarded wird, muss das Forwarding in der LINUX-Kernel-Parameter-Konfigurations-datei aktiviert werden.

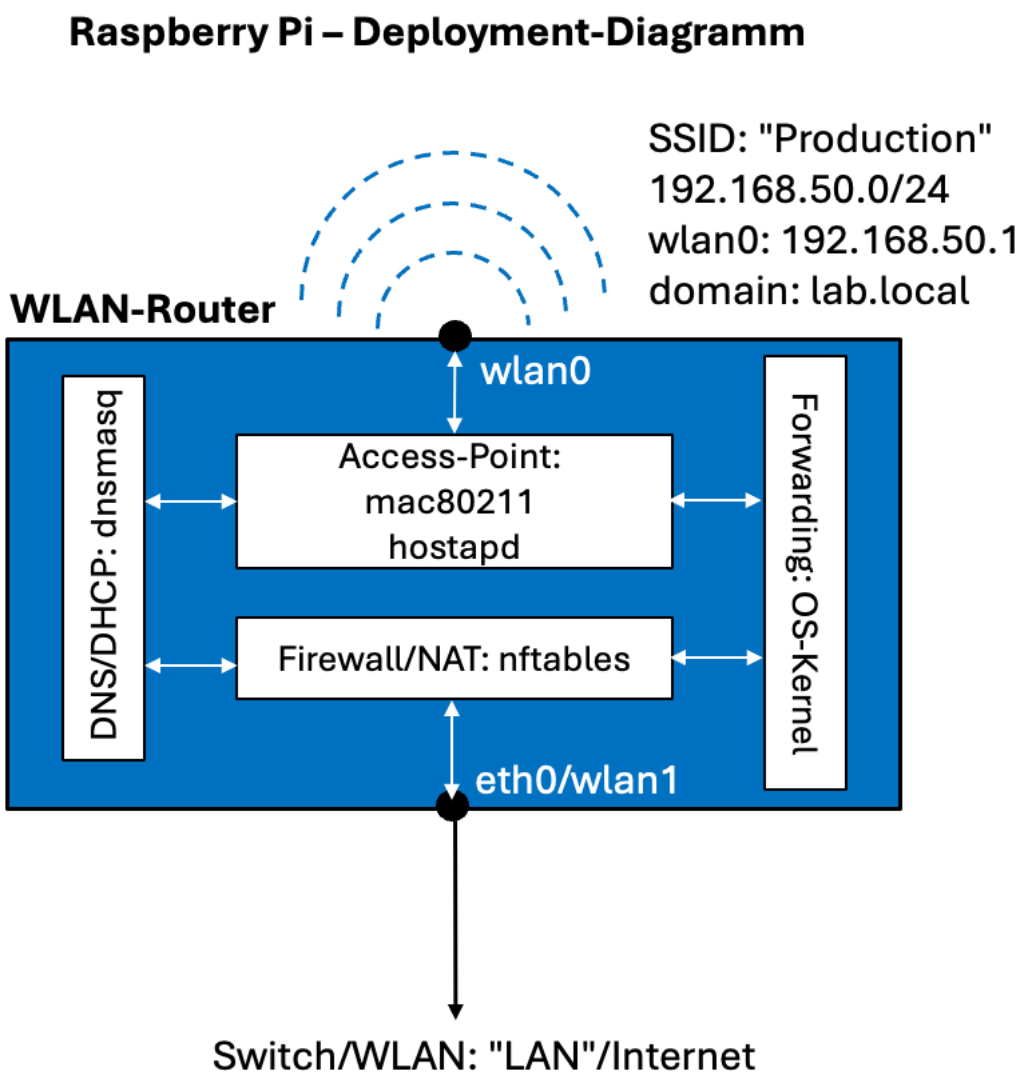
```
$sudo nano /etc/sysctl.conf

#Aktivieren von ipv4 forwarding
net.ipv4.ip_forward=1
```

- Der LINUX-Kernel verwendet dann seine lokale Routing-Tabelle, um Pakete an das richtige Interface weiterzuleiten.
- Die lokale Routing-Tabelle eines Hosts (z. B. Raspberry Pi) wird automatisch durch den Kernel/Netzwerkstack gefüllt.
  - Sobald eine Netzwerkinterface mit einer IP-Adresse und Subnetz konfiguriert wird, wird ein entsprechender connected route Eintrag ergänzt.
  - Besitzt ein Interface ein Default-Gateway, wird für dieses ein entsprechender Default-Route Eintrag erstellt.
- Die lokale Routing-Tabelle kann wie folgt angezeigt werden und sollte für den WLAN-AP mindestens 2 Einträge besitzen
  - 1x Default-Route für das LAN-Interface eth0
  - 1 x Connected Route für das Interfaces wlan0

WLAN-AP (wlan0) → IP: 192.168.50.1 , Netz: 192.168.50.0/24
Ethernet (eth0) → IP: 192.168.178.20 , Netz: 192.168.178.0/24
Internet-Gateway → IP: 192.168.178.1

```
$ip route

default via 192.168.178.1 dev eth0
192.168.178.0/24 dev eth0 proto kernel scope link src 192.168.178.20
192.168.50.0/24  dev wlan0 proto kernel scope link src 192.168.50.1
```

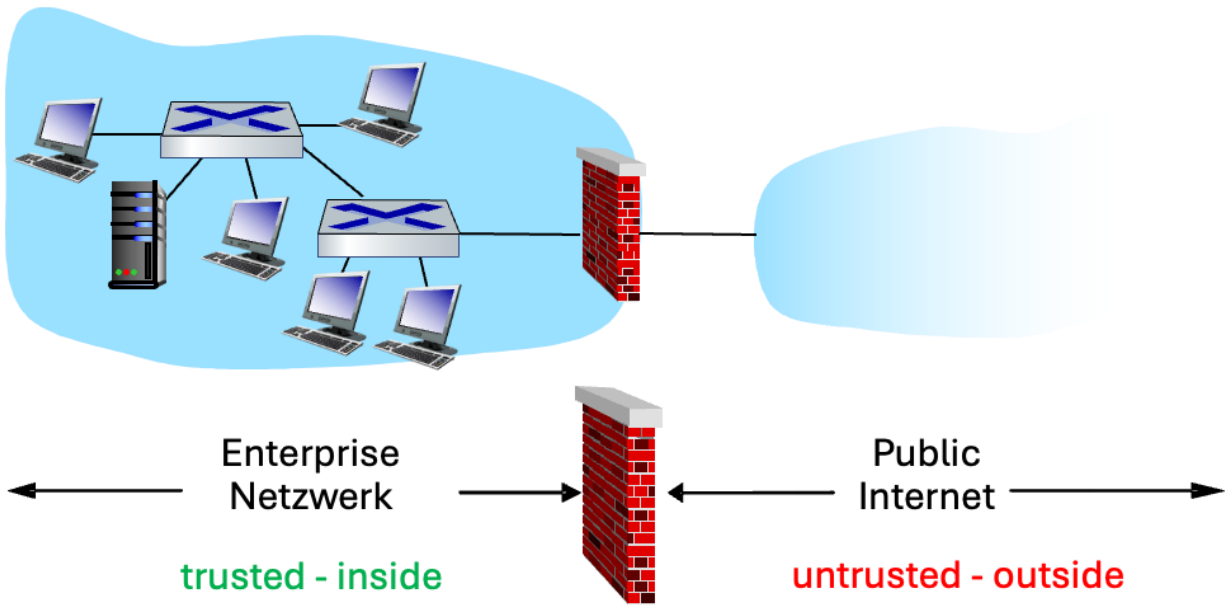
2.5 Firewall


Raspberry Pi – Deployment-Diagramm
SSID: "Production"
192.168.50.0/24
wlan0: 192.168.50.1
domain: lab.local
WLAN-Router
wlan0
DNS/DHCP: dnsmasq
Access-Point:
mac80211
hostapd
Firewall/NAT: nftables
Forwarding: OS-Kernel
eth0/wlan1
Switch/WLAN: "LAN"/Internet

Firewall – Grundkonzept einer Edge-Firewall

Eine Firewall ist ein Host, der den Zugriff auf ein Netzwerk vermittelt und bestimmte Zugriffsarten auf der Grundlage einer konfigurierten Sicherheitsrichtlinie erlaubt oder verbietet.

- Grundidee: Schutz eines vertrauenswürdigen Netzwerkes vor einem nicht vertrauenswürdigen Netzwerk durch ein definiertes Regelwerk.
- Inline: Der gesamte Datenverkehr wird von innen (Unternehmensnetzwerk) nach außen (Internet) und umgekehrt durch die Firewall geleitet werden.
- Nur autorisierter Datenverkehr, wie in der lokalen Sicherheitsrichtlinie der Firewall definiert, wird weitergeleitet werden.
- Firewalls, die sich am Übergang vom Unternehmensnetzwerk in das Internet befinden, werden auch als Edge-Firewalls bezeichnet.



- Die Firewall selbst ist gehärtet, so das Sie resistent gegen Angriffe oder unerlaubte Eindringen von außen ist:
  - gehärtetes Betriebssystem mittels eines Secure SDLC entwickelt,
  - gehärtete Hardware.

Stateless und Stateful Firewalls

Paketfilter-Firewall (Stateless / Zustandslose Firewall)

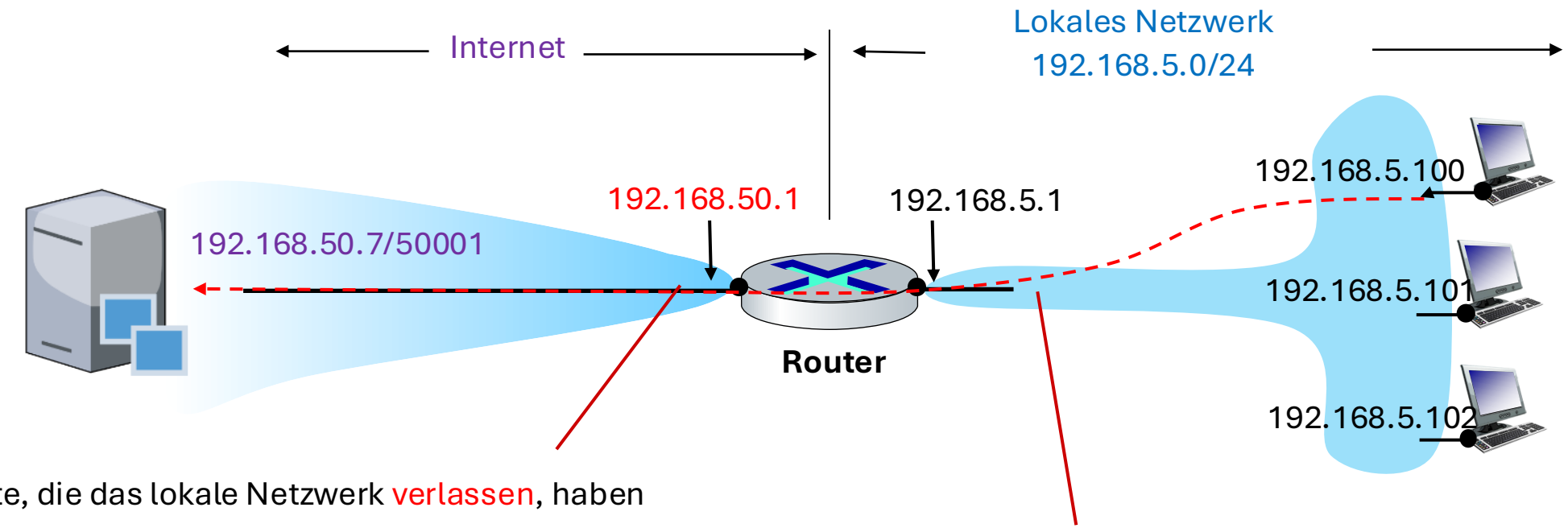
- Funktionsweise: Einfacher Vorabfilter
  - FW untersucht den Header jedes Datenpakets isoliert (Quell-/Ziel-IP-Adresse, Portnummer, Protokoll).
  - FW vergleicht die Headerwerte mit einer Reihe vordefinierter Regeln in Form einer Access Control List (ACL), um das Paket zu erlauben oder zu blockieren.
- Vorteile: Sehr schnell, geringer Ressourcenverbrauch, einfach zu implementieren.
- Nachteile:
  - Kann den Zustand einer Verbindung nicht nachverfolgen, was sie anfällig für komplexere Angriffe macht (z.B. IP-Spoofing).
  - Versteht den Kontext einer Kommunikation nicht, führt zu Regeln die weite Port-Bereiche öffnen.

Stateful Inspection Firewall (Zustandsbehaftete Firewall)

- Funktionsweise: Filtert auf Basis von Protokollzuständen
  - FW prüft nicht nur Paket-Header, sondern verfolgt auch den Status aktiver Verbindungen.
  - FW verwaltet eine Tabelle aller offenen Verbindungen und erlaubt nur Datenverkehr, der Teil einer legitimen, bestehenden Sitzung oder einer gültigen neuen internen Anfrage ist.
- Vorteile: Deutlich sicherer als zustandslose Filterung. Blockiert unerwünscht eingehenden Verkehr effektiv und kann Angriffsmuster erkennen.
- Nachteile: Ressourcenintensiver als Paketfilterung, was die Leistung in sehr hochvolumigen Netzwerken beeinträchtigen kann.

NAT: Funktionsweise

- NAT: Nicht öffentliche IP-Adressen der Geräte im lokalen Netzwerk werden auf einen Pool von öffentlichen Adressen gemapped (Bezeichnung: Overloading NAT).



Alle Pakete, die das lokale Netzwerk verlassen, haben dieselbe öffentliche Outside-NAT-IP-Adresse: 138.76.29.7, jedoch unterschiedliche Outside-TCP-Portnummern

Pakete mit Sender oder Empfänger in diesem Netzwerk haben 192.168.5.0/24 als Adresse für diesen Sender/Empfänger.

Firewall: nftables

- Die nftables-Firewall ist eine Stateful-Firewall und in den meisten LINUX-Derivaten standardmäßig schon installiert.
- nftables ist keine eigenständige Anwendung, sondern ist ein Teil des LINUX-Kernels.
- Die nftables-Regeln werden in der Konfigurationsdatei `nftables.conf` oder über das Tool `nft` konfiguriert.

```
$sudo nano /etc/nftables.conf
```

- Diese Regeln werden auf eingehende (INPUT), ausgehende (OUTPUT) und weitergeleitete (FORWARD) Datenpakete angewandt werden.
- Der Linux-Kernel hat bestimmte Fixpunkte – die Hooks, an denen Netzwerkpakete „abgefangen“ werden können.
- Die Firewall-Regeln werden an diesen Hooks in den Kernel geladen (eingehängt).
  - Trifft ein neues Paket ein, durchläuft der Kernel die Hooks in einer festen Reihenfolge, und führt die dort eingehängten nftables-Regeln aus.
- Eine **nftables-Konfiguration** besteht aus:
  - Tables: inet filter, ip nat

    Logische Sammlung von Chains

    inet filter: Firewall

    ip nat: NATTING
  - Chains (z. B. input, output, forward)

    Kette an Regeln für einen Verarbeitungsschritt, die an einen bestimmten "Hook" gehängt werden.
  - Rules (z. B. „erlaube SSH“)

    Einzelne Filterregeln pro Verarbeitungsschritt innerhalb einer Chain.

Konfiguration von nftables

- Für unser Projekt benötigen wir 2 Typen von **Tabellen**:
  - `table inet filter {…}`: Regelwerk für das Filtern von IP4 & IP6 Paketen
  - `table ip nat {…}`: NAT-Regeln für den IP4-Datenverkehr
- Innerhalb der Tabelle `inet filter` verwendet man die folgenden Chains:

Hook / Typ	**Beschreibung**
`chain input {…}`	Eingehende Pakete für den WLAN-AP
`chain output {…}`	Ausgehende Pakete vom WLAN-AP
`chain forward {…}`	Pakete, die durch den WLAN-AP geroutet werden sollen.

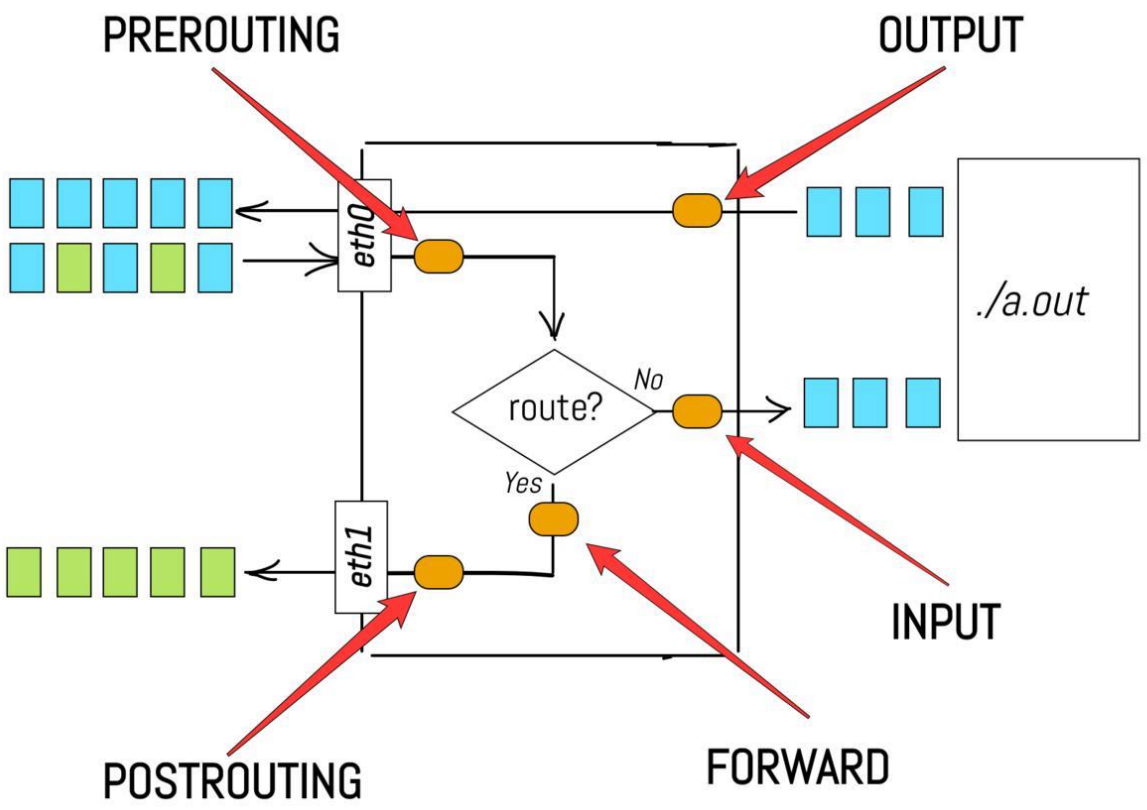
- Innerhalb der Tabelle `ip nat` verwendet man die folgenden Chains:

Hook / Typ	**Beschreibung**
`chain prerouting {…}`	Eingehende Pakete vor internem Routing.
`chain postrouting {…}`	Ausgehende Pakete nach internem Routing

iif	incoming interface	Interface, über das ein Paket **ankommt**
oif	outgoing interface	Interface, über das ein Paket **gesendet wird**

Hooks im Linux Kernel

Hook	Bedeutung
PREROUTING	Paket kommt rein (noch bevor Routing entschieden wird)
INPUT	Paket ist für den lokalen Rechner bestimmt
FORWARD	Paket wird durch den Rechner weitergeleitet
OUTPUT	Paket wird lokal erzeugt
POSTROUTING	Paket verlässt den Rechner



Chain Input: Typisches Regelwerk

- **type filter hook input priority 0;**
  - verbindet die chain input mit dem Input-Hook des Netfilter-Stacks, chain wird mit Standard-Priorität 0 ausgeführt.
  - filtert Pakete, die an den WLAN-AP gehen
- **policy drop;**
  - Default-Deny: alles wird verworfen, wenn keine Regel passt
  - Nur explizit erlaubter Verkehr wird durchgelassen.
- **iif "lo" accept**
  - erlaubt Traffic über das Loopback-Interface (localhost)
  - WLAN-AP interne Kommunikation
- **ct state established,related accept**
  - `ct` = connection tracking
  - `state established, related:` erlaubt Paktete zu einer bereits bestehenden Verbindung & indirekte Pakete (ICMP-Nachrichten zu einer erlaubten Nachricht)
- **iif "wlan0" tcp dport 22 accept**
  - SSH-Verbindung: erlaubt über das Interface wlan0 eines TCP-Verbindung auf Destination Port 22

```
table inet filter {
 chain input {
   # Hängt die Chain "input" an den Hook "input"
   type filter hook input priority 0;
   # Default-Deny-Policy
   policy drop;
   # Interne Kommunikation über Interface Loopback (lo)
   # immer erlauben
   iif "lo" accept
   # Bereits bestehende oder verwandte Verbindungen
   # erlauben
   ct state established,related accept
   # SSH vom IoT-Netz (Interface wlan0) erlauben
   iif "wlan0" tcp dport 22 accept
   # ICMP (Ping, …) erlauben
   ip protocol icmp accept
 }
…
   }
```

Chains for Natting

- *chain prerouting*: Diese Chain verarbeitet *eingehende Pakete* und verändert den Destination Port **(Zieladressen / Port-Forwarding)** durch den internen Port aus der NAT-Tabelle.
  - Automatischen Zuordnung der Pakete zum Ziel erfolgt über die vom WLAN-AP automatisch konfigurierte NAT-Tabelle.
  - *priority dstnat*: Standardpriorität für Destination NAT (DNAT)
- *chain postrouting:* Diese Chain wird nach der Routing-Entscheidung, aber vor dem Versenden der Pakete angewandt.
  - Typisch für Source NAT (SNAT) oder Masquerading.
  - *oif "wlan1" masquerade*:

    IP Adress Translation: Wenn ein Paket über das Interface wlan1 nach draußen geht, wird die Quell-IP gemapped auf die IP von wlan1.

    Port Adress Translation: Der Quellport wird automatisch auf einen freien Port des ephermal-Bereiches (>32768) gemapped.

```
table ip nat {
    chain prerouting {
        #Hook für prerouting
        type nat hook prerouting priority dstnat;
        #Eingehende Pakete werden akzeptiert
        policy accept;
    }
    chain postrouting {
        #Hook für prerouting
        type nat hook postrouting priority srcnat;
        policy accept;
        #Ausgehende Pakete auf wlan1 werden genattet
        oif "wlan1" masquerade
    }
}
```

Firewall: nftables

- Nach dem Setzen der Regeln, kann die Syntax der Konfiguration auf Korrektheit überprüft werden

```
$ sudo nft -c -f /etc/nftables.conf


  -c = Check-Only (führt einen Syntax-Check
       durch, ohne die Regeln zu aktivieren.)
```

- Anschließend können die Regeln per Restart des nftables Service geladen werden

```
#Aktivieren des automat. Starts von nftables
#beim Reboot
$ sudo systemctl enable nftables
#Restart des Services nftables, liest die
#Konfigdatei neu
$ sudo systemctl restart nftables
```

Firewall: nftables

- Den Status der neu gestarteten Firewall kann wie folgt überprüft werden:

```
$sudo systemctl status -l nftables
$sudo journalctl -e -u nftables
```

- Anzeigen der aktiven Regeln mit dem Befehl

```
$sudo nft list ruleset
```

- Da die Firewall-Regeln, sich auf die Host-Interfaces (wlan0) beziehen, muss beim System-Neustart sichergestellt werden, das diese schon aktiv sind.

- Das kann in der Unit-Datei für `nftables` konfiguriert werden.

```
$sudo systemctl edit nftables
```

- Dort muss der folgende Inhalt ergänzt werden:

```
[Unit]
Description=Load nftables rules after network
After=network-online.target
Wants=network-online.target
```

Protokollierung der "Drops"

- Um zu sehen welche Pakete die Firewall "dropped" kann beispielsweise in der Input-Chain die folgende Regel am Ende aufgenommen werden.

```
# Loggen der zu droppenden Pakete


log prefix "Dropped packet: " flags all drop
```

- `log`: Aktiviert das Logging für das Paket. Jedes Paket, das diese Regel trifft, wird ins systemd-journald geschrieben.
- `prefix "Dropped packet": "` Fügt jedem Logeintrag diesen Text voran.
- `flags all`: Loggt alle verfügbaren Informationen zum Paket: Quell- und Ziel-IP, Ports, Protokoll, Interface, TCP-Flags usw.
- `drop`: Danach wird das Paket verworfen.

- Die gedroppten Pakete können dann wie folgt angezeigt werden:

```
$sudo journalctl -k -n 80 | grep "Dropped packet"



-k   :  nur Kernel-Meldungen anzeigen (z. B.
        Treiber, Netzwerk, Firewall).
-n 80:  die letzten 80 Einträge
| grep "Dropped packet": die den Text "Dropped
packet" enthalten
```

Größe der Logfiles in journalctl

- Um zu verhindern, das die Log-Files des systemd-Journals zu groß werden und um Performance senkende Schreibzugriffe auf die SD-Karte (schont SD-Karte) zu vermeiden, kann das folgende in `/etc/systemd/journald.conf` definiert werden :

```
[Journal]
Storage=volatile
RuntimeMaxUse=100M
RuntimeKeepFree=20M
RuntimeMaxFileSize=10M
```

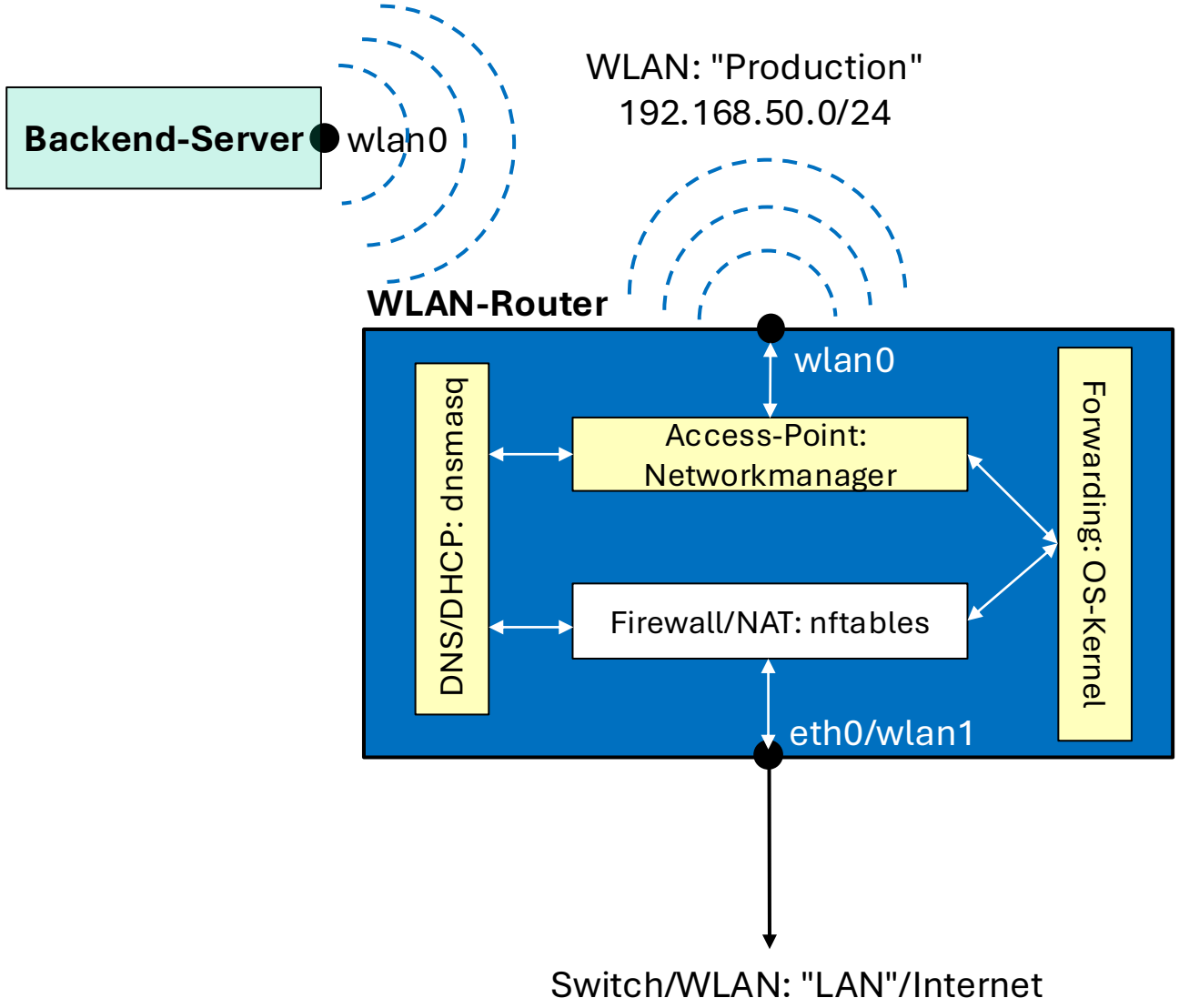
  - `Storage=volatile`: Logs werden im RAM auf dem /run (=tmpfs-Filesystem im RAM) gespeichert und sind nach einem Neustart weg.
  - `RuntimeMaxUse=100M`: Maximale Größe der Log-Dateien im RAM beträgt 100MB.
  - `RuntimeKeepFree=20M`: Speicherplatz der für andere Prozesse auf /run frei bleiben muss.
  - `RuntimeMaxFileSize=10M`: maximale Größe eines Log-Files (➞ 10 Log-Files in /run)
- Nach der Konfiguration sollte der systemd-journald Service wie folgt neu gestartet werden

```
$sudo systemctl restart systemd-journald
```

- /run wird für **kritische Systemdaten** gebraucht
  - PID-Dateien in /run/systemd/
  - Sockets in /run/dbus/
  - Lockfiles
  - temporäre Configs
- Die Größe und der verwendete und freie Speicherplatz von /run kann wie folgt angezeigt werden (df : disk free)

```
$df -h /run
```

2.6 Backend-Server Integration

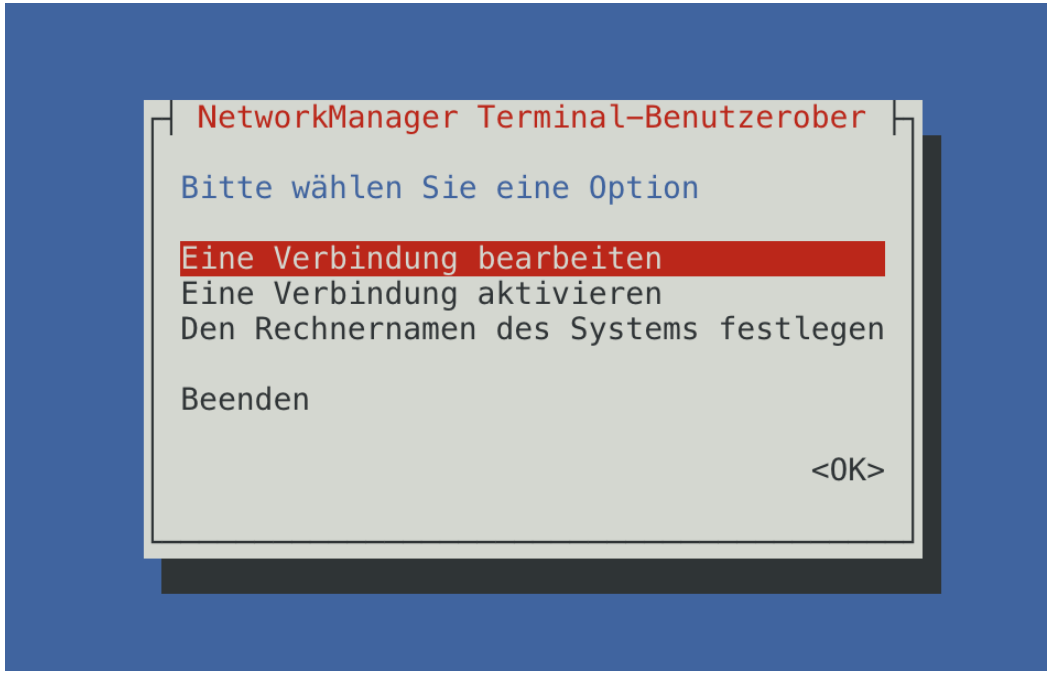


Integration Backend-Server

- Der Raspberry PI Backend Server wird als WLAN-Client im Production-Netz eingerichtet.
- Dazu kann das grafische Benutzer-Interface des Networkmanager verwendet werden.

```
$sudo nmtui
```

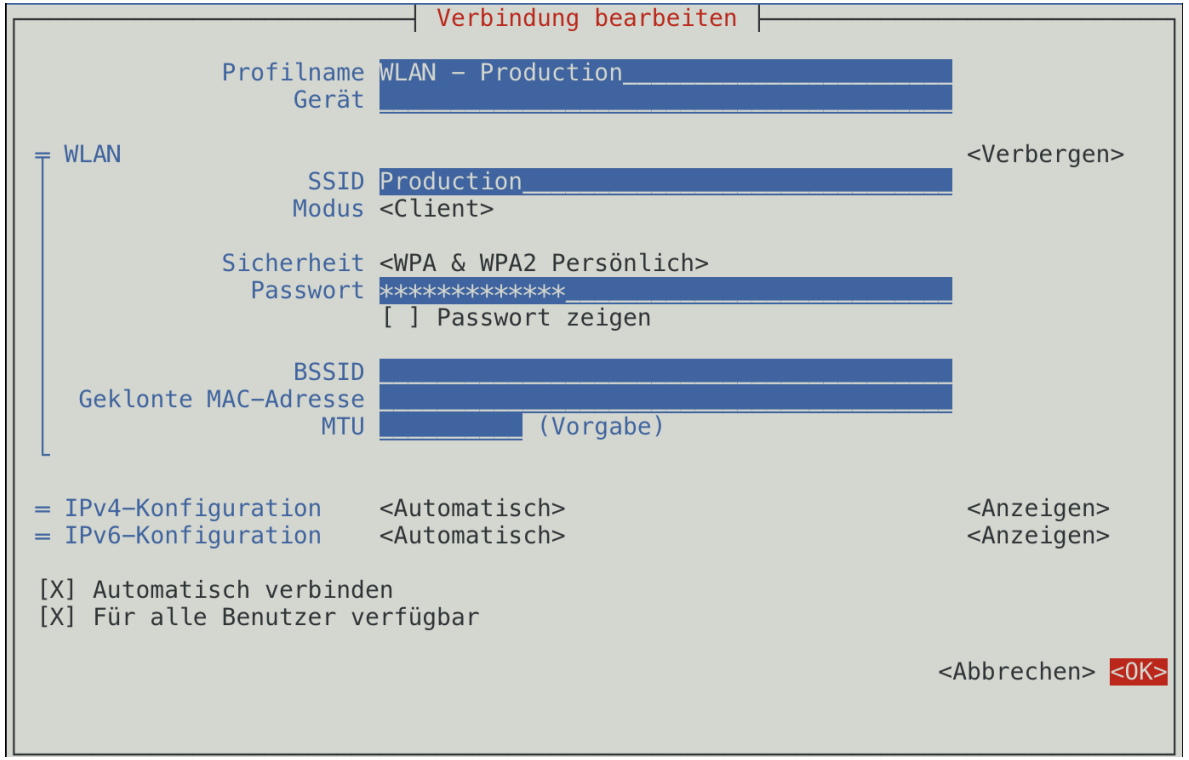
- Eine Verbindung verarbeiten auswählen:



- Anschließend wählen Sie im gezeigten Menü:

  `Hinzufügen/WLAN/Erstellen`

  - `Profilname: WLAN-Production`
  - `SSID: Production`
  - `Passwort: ************`



Verbindungsstatus prüfen

- Status der WLAN-Verbindung prüfen

```
$sudo nmcli device status

DEVICE     TYPE       STATE       CONNECTION
wlan0      wifi       verbunden   WLAN - Production
```

- Sollte der Status auf nicht verbunden stehen, können Sie einen Reconnect versuchen

```
$sudo nmcli device wifi connect "Production"
                        password "PASSWORT"
```

- Im erneuten Fehlerfall Logs analysieren

```
$journalctl -u NetworkManager -n 50
```

- Testen der IP-Konfiguration. IP-Adresse (inet) und Broadcast-Adresse (brd) sollte aus dem konfigurierten DHCP-Bereich stammen.
  - brd-Adresse ist die größte Adresse im konfigurierten Subnetz: /24 → .255

```
$ip a show wlan0

3: wlan0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
pfifo_fast state UP group default qlen 1000
    link/ether e4:5f:01:00:bd:c6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.30/24 brd 192.168.50.255 scope global
    […]
```

Verbindungsstatus prüfen

- Wurde die Gateway – Information korrekt übermittelt. Wird diese per DHCP übermittelt, so wird Sie im Arbeitsspeicher des PI in der Routing-Tabelle gespeichert

```
$ip route

default via 192.168.50.1 dev wlan0 proto dhcp src 192.168.50.30 metric 600
192.168.50.0/24 dev wlan0 proto kernel scope link src 192.168.50.30 metric 600
```

  - `default:` Das Ziel ist das "Standard-Gateway" (0.0.0.0/0), also alles, was nicht für das lokale Netzwerk bestimmt ist.
  - `via 192.168.50.1`: Sende Pakete für dieses Ziel an den Router mit der IP 192.168.50.1 (=WLAN-AP)
  - `dev wlan0`: Verwende die Netzwerkschnittstelle wlan0.
  - `proto dhcp`: Diese Route wurde automatisch vom DHCP-Server bezogen.
  - `src 192.168.50.30`: IP-Adresse des Backend-Servers
  - `metric 600`: Die "Kosten" dieser Route, wird für WLAN-Verbindungen auf diesen Wert gesetzt.

Verbindungsstatus prüfen

- Ist das Gateway (=AP) per IP erreichbar

```
$ping -c 4 192.168.50.1

PING 192.168.5.1 (192.168.5.1) 56(84) bytes of data.
64 bytes from 192.168.5.1: icmp_seq=1 ttl=64 time=1.59 ms
64 bytes from 192.168.5.1: icmp_seq=2 ttl=64 time=7.81 ms
64 bytes from 192.168.5.1: icmp_seq=3 ttl=64 time=7.82 ms
64 bytes from 192.168.5.1: icmp_seq=4 ttl=64 time=7.81 ms
```

- Wurde der DNS-Server richtig konfiguriert. Auch der DNS-Server wird dynamisch im RAM des NetworkManager gespeichert. Dieser Wert kann wie folgt beim NetworkManager abgefragt werden:

```
$sudo nmcli dev show wlan0 | grep DNS
IP4.DNS[1]:                                192.168.50.1
```

- Funktioniert die Namensauflösung für Hosts (z.B.: www.google.de) und das Routing ins Internet:

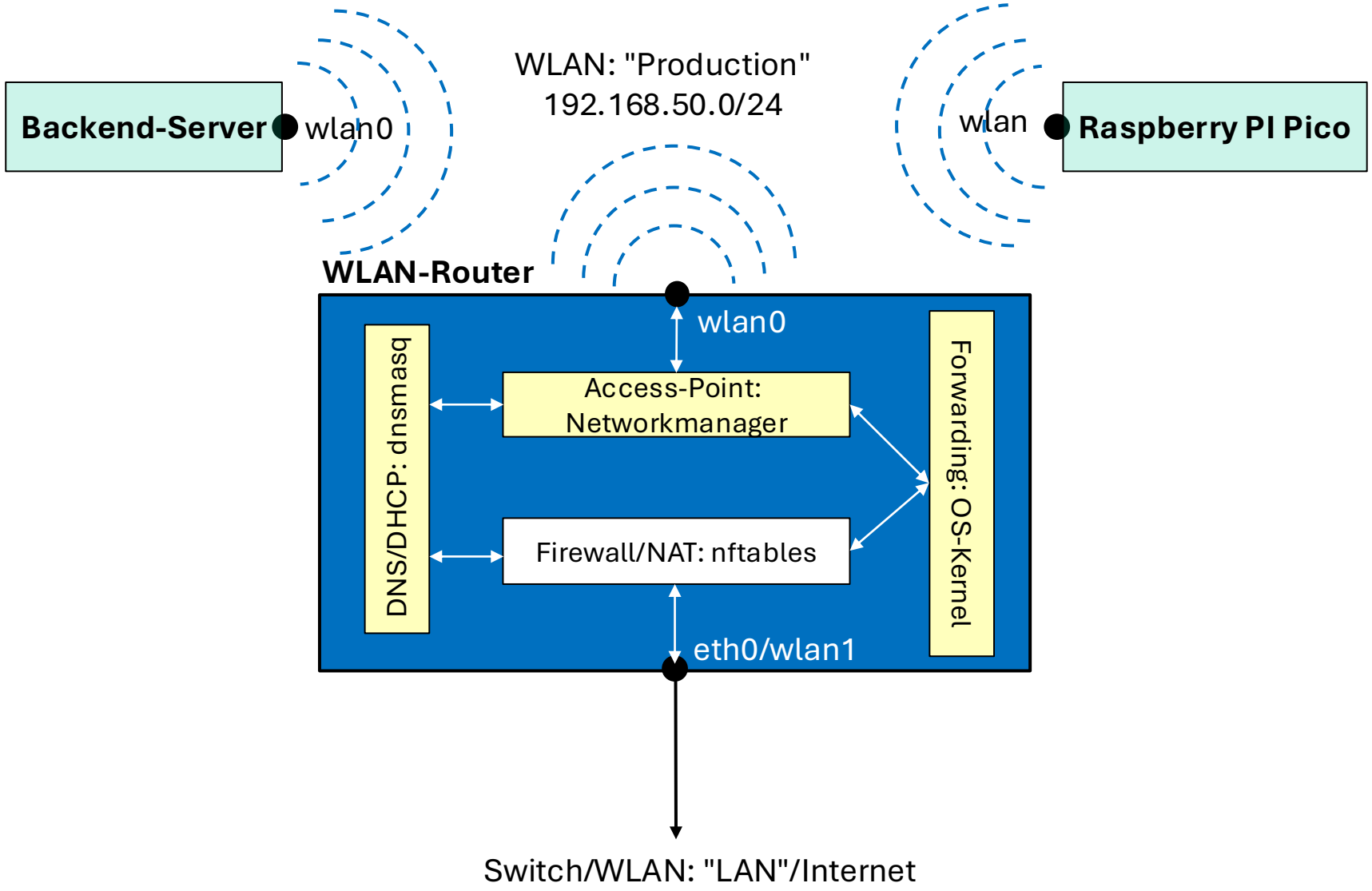
```
$ping -c 4 www.google.com

PING www.google.com (142.250.184.196) 56(84) bytes of data.
64 bytes from fra24s11-in-f4.1e100.net (142.250.184.196):
icmp_seq=1 ttl=118 time=16.5 ms
64 bytes from fra24s11-in-f4.1e100.net (142.250.184.196):
icmp_seq=2 ttl=118 time=21.8 ms
64 bytes from fra24s11-in-f4.1e100.net (142.250.184.196):
icmp_seq=3 ttl=118 time=21.6 ms
64 bytes from fra24s11-in-f4.1e100.net (142.250.184.196):
icmp_seq=4 ttl=118 time=60.3 ms
```

Verbindungsstatus prüfen

- Die gesamte vom NetworkManager verwaltete Information zu der wlan-Verbindung WLAN - Production erhält man mittels

```
$sudo nmcli dev show wlan0
GENERAL.DEVICE:                         wlan0
GENERAL.TYPE:                           wifi
GENERAL.HWADDR:                         E4:5F:01:00:BD:C6
GENERAL.MTU:                            1500
GENERAL.STATE:                          100 (verbunden)
GENERAL.CONNECTION:                     WLAN - Production
GENERAL.CON-PATH:                       /org/freedesktop/NetworkManager/ActiveConnection/6
IP4.ADDRESS[1]:                         192.168.5.30/24
IP4.GATEWAY:                            192.168.5.1
IP4.ROUTE[1]:                           dst = 192.168.5.0/24, nh = 0.0.0.0, mt = 600
IP4.ROUTE[2]:                           dst = 0.0.0.0/0, nh = 192.168.5.1, mt = 600
IP4.DNS[1]:                             192.168.5.1
IP4.DOMAIN[1]:                          iot.local
```

2.7 Mikrocontroller Integration



Permanenter Speicher im PI Pico

- **Permanenter Speicher im Raspberry Pi Pico:** Der RP2040-Mikrocontroller selbst hat keinen internen Flash, sondern verwendet externen Quad Serial Peripheral Interface Flash (QSPI)-Flash (2 MB) auf dem Board.
- QSPI verwendet 4 Datenleitungen für das Lesen und Schreiben.
- XIP (Execute-In-Place): Der Prozessor kann Code direkt aus dem QSPI-Flash ausführen, ohne ihn zuvor in den RAM laden zu müssen.

```
RPI_PICO_W-….uf2 MicroPython-Firmware
│
├─ Bootcode
├─ MicroPython Interpreter
├─ Hardware-Abstraktion (GPIO,...)
└─ REPL (interaktive Console)
```

REPL (Read-Eval-Print Loop)

- Im Flash-Speicher werden die Firmware und die Programme für den PI Pico permanent gespeichert:
  - MicroPython-Firmware: `RPI_PICO_W-….uf2`
  - main.py: Python-Skript das automatisch ausgeführt wird
  - Zertifikate: TLS-Session-Etablierung
  - Daten: Von einem Programm gespeicherte Daten

Eine UF2-Datei (USB Flashing Format v2) ist ein spezielles Firmware-Format, das u. a. beim Raspberry Pi Pico und anderen Microcontrollern verwendet wird, um Firmware einfach auf den Chip aufzuspielen.

Inbetriebnahme Raspberry Pi Pico

- Der Raspberry Pi Pico wird nicht wie ein normaler PC „installiert“, sondern man schreibt Programme direkt – also ohne Betriebssystem – in den Flash-Speicher des Mikrocontrollers.
- Zur Programmierung des Raspberry Picos stehen 2 Programmier-sprachen zur Verfügung: MicroPython und C/C++
- Im Rahmen dieser Vorlesung wird MicroPython verwendet.

Setup der MicroPython Umgebung

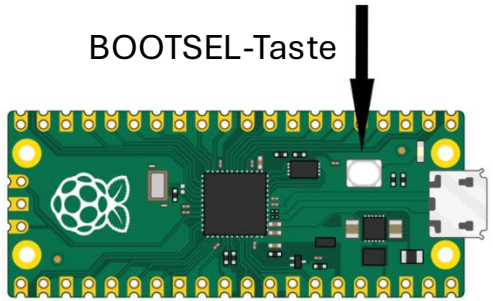
- Eine übersichtliche Anleitung zum Setup eines PI Picos finden Sie unter

  https://projects.raspberrypi.org/en/projects/getting-started-with-the-pico/3

Inbetriebnahme Raspberry Pi Pico

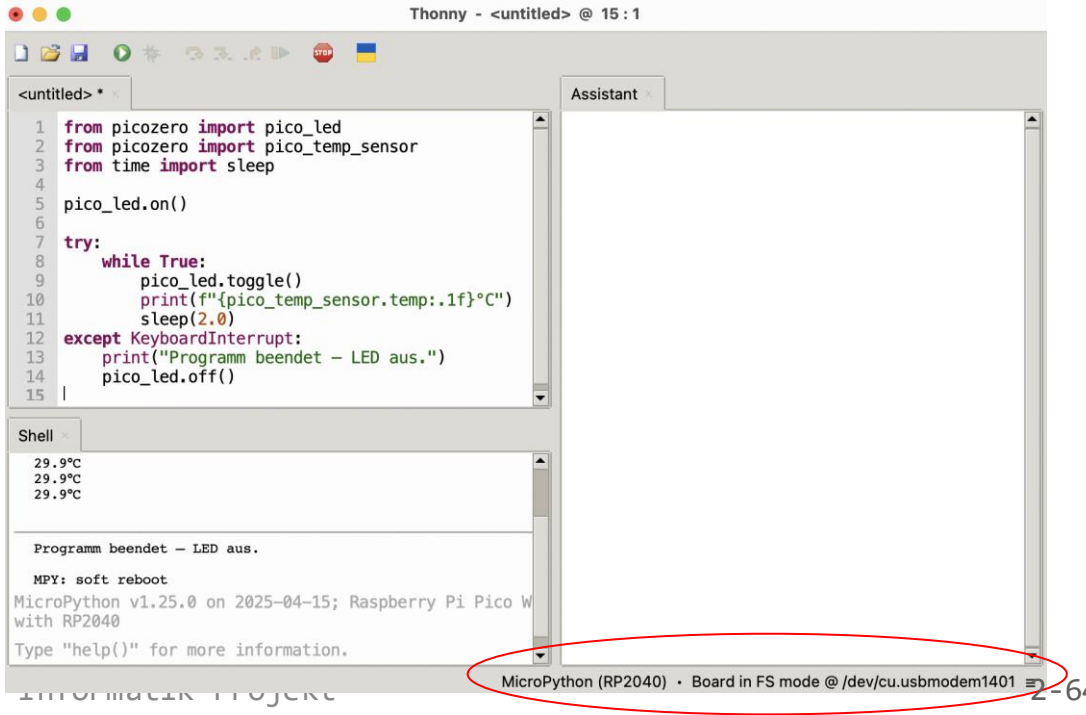
Firmware auf den Pico spielen

- Die offizielle MicroPython-Firmware für den Pico kann unter der folgenden URL Pico W herunterladen werden.
  - Pico mit gedrückter BOOTSEL-Taste (weiße Taste auf Pico) an den Entwicklungs-PC per USB-Kabel anschließen.
  - Pico erscheint als USB-Laufwerk "RPI-RP2" auf dem Entwicklungs-PC
  - Die .uf2-Datei mit der MicroPython-Firmware auf das Laufwerk "RPI-RP2" per Drag und Drop kopieren.
  - Pico startet neu.
  - Fertig!



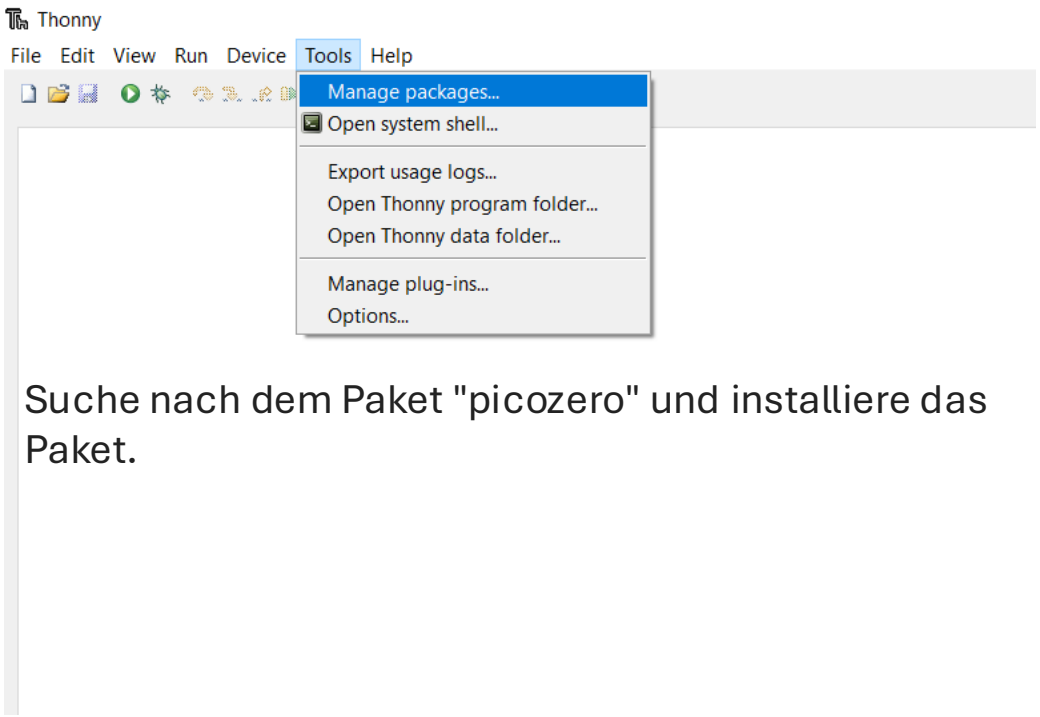
Code schreiben

- Kostenfreien Editor Thonny auf seinen Entwicklungs-PC installieren.
- Thonny starten und als Interpreter MicroPython (RP2040) im Editormenü auswählen.
- Thonny mit der richtigen seriellen Schnittstelle verbinden (Bottom-Menüleiste, rechts)



Initialer Funktionstest des PI Picos

- Um die Funktion des PI Pico und seine Integration in die Entwicklungsumgebung auf dem PC zu testen, entwickeln Sie ein "Hello World" – Python Programm für den PI Pico.
- **Anforderung:**
  - Onboard-LED soll alle 2 Sekunden ein-/ausgeschaltet werden.
  - Onboard-Temperatur Sensor soll alle 2 Sekunden die aktuelle Temperatur ausgeben.
  - Wird das Programm per Keyboardtaste beendet, wird die Onboard-LED ausgeschaltet.
- **Tips:**
  - Starten Sie den Thonny-Editor und installiere Sie sich über das Tool-Menü (siehe Abbildung rechts) die Bibliothek "picozero" in den Flash-Speicher des Pico.
  - "picozero" erhält eine Vielzahl an Funktionen, um IoT-Geräte zu steuern.



Suche nach dem Paket "picozero" und installiere das Paket.

- **Bedienung Thonny:**
  - grüner Pfeil - Button: starten des Python-Programms
  - roter Stop - Button: beenden des Python Programm

Pi Pico Integration ins WLAN

- Um die Integration des PI Picos in das WLAN-Production zu testen, inklusive der Anbindung an den DHCP- und DNS-Server, erstellen Sie eine Python-Applikation mit den folgenden funktionalen Requirements:
  - Pico W verbindet sich ins WLAN "Production"
  - Programm holt all 10 Minuten die aktuelle Temperatur für Friedrichshafen (latitude=47.65 ; longitude=9.48) von der folgenden RESTful API im Internet https://api.open-meteo.com/v1/forecast?latitude=47.65&longitude=9.48&current_weather=true
  - LED blinkt abhängig von der Temperatur:

    🌨 kalt (<10 °C) → langsames Blinken (alle 2 s)

    🌤 mild (10–25 °C) → mittleres Blinken (1 s)

    🌞 heiß (>25 °C) → schnelles Blinken (0.3 s)

- Zur Umsetzung des Programmes benötigen sie die folgenden Bibliotheken:
  - network: Funktionen zur Steuerung des WLAN-Interfaces
  - urequest: Funktionen zum Senden und Empfangen von HTTP-Nachrichten.

    "u": steht für Mikro und deutet auf eine im Funktions-umfang reduzierte Bibliothek hin

    "request": HTTP-Bibliothek aus der Python-Welt
  - time: Funktionen der Zeit (warten, Uhrzeit ausgeben, ...)

Projekt: Integration des PI Picos in das WLAN-Production

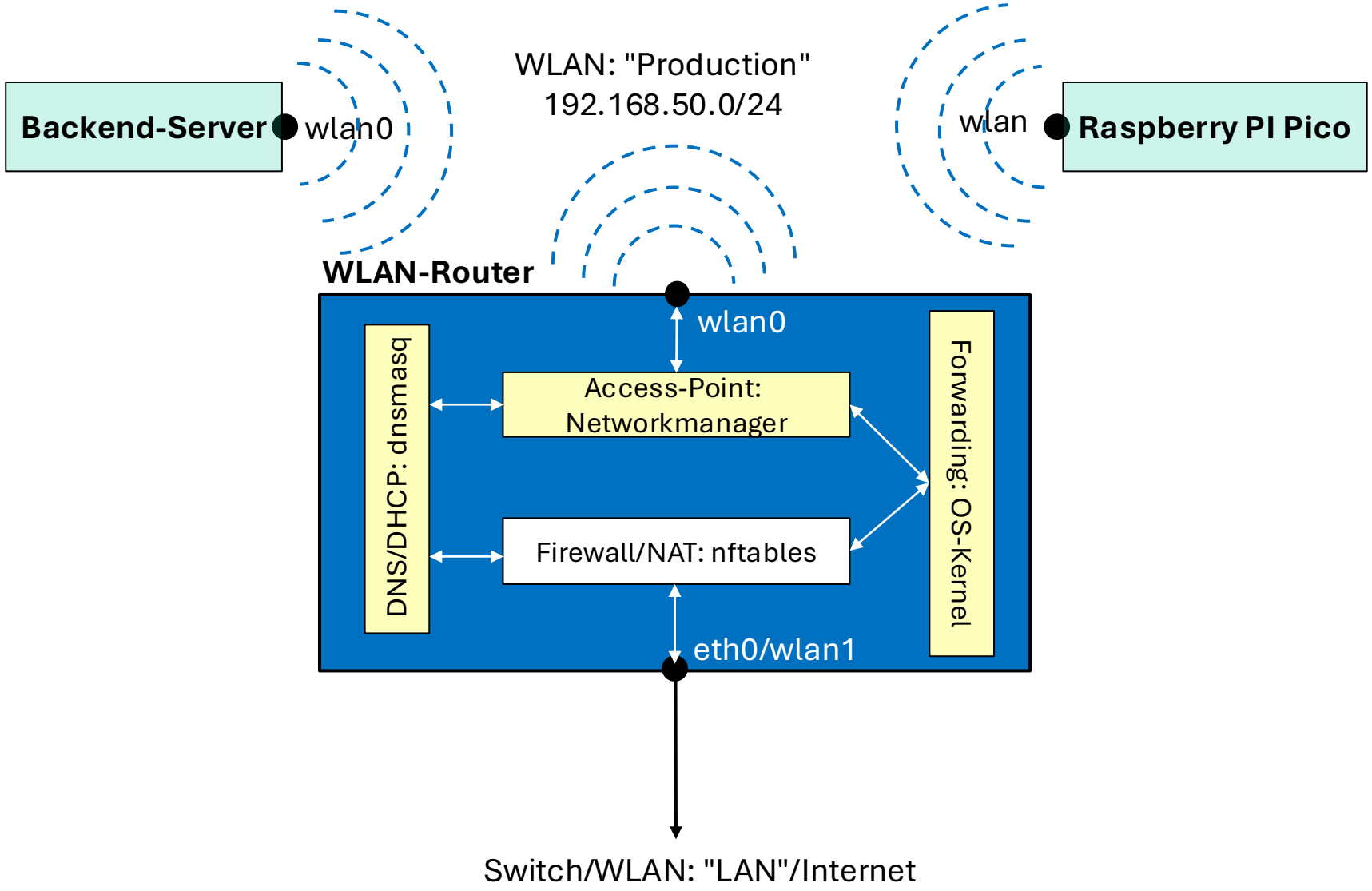
- Erstellen Sie das zuvor beschriebene Programm in MikroPython.
- Dokumentieren Sie ihr Programm und das erreichte Ergebnis.
- Wenn Sie Temperaturdaten über die RESTful API erhalten, ist die Integration des PI Picos vollständig erfüllt.

Autostart eines entwickelten MicroPython-Programmes

- Programm in Thonny schreiben und testen
- Auf dem Pico die Python-Datei speichern
  - In Thonny: Datei → Speichern unter… → Raspberry Pi Pico
  - Dateiname main.py (klein geschrieben) wählen.
  - Dadurch überschreibst du evtl. vorhandenes main.py.
- Pico neu starten
  - USB-Kabel trennen und wieder anstecken oder Reset-Taste drücken.
  - Beim Start führt MicroPython automatisch main.py aus.
- Falls das main.py in einer Endlosschleife hängt und per Thonny REPL nicht erreichbar ist:
  - **BOOTSEL-Taste gedrückt halten** und Pico anstecken → er erscheint als USB-Laufwerk RPI-RP2.
  - main.py löschen oder umbenennen.
  - Pico neu starten.

- **REPL** = **R**ead **E**val **P**rint **L**oop
  - Lese Eingaben → Führe sie aus → Zeige das Ergebnis → Warte auf die nächste Eingabe.

2.8 Projektarbeit



Aufgabe:

- Aktualisieren Sie ihren Projektplan: Ergänzen Sie diesen um Arbeitspakete, deren Abarbeitungsreihenfolge und dem für die Umsetzung jeweils verantwortlichen Teammitglied.
- Aktualisieren Sie ihr Deployment-Diagramm.
- Überprüfen Sie ob ihr Architekturdesign die fundamentalen Sicherheitsprinzipien erfüllt. Ergänzen Sie aus ihrer Sicht fehlende Eigenschaften.
- Konfigurieren Sie anschließend den WLAN-AP inklusive dem dnsmasq-Service.
  Entwickeln Sie hierzu ein Python-Programm, dass die Konfiguration automatisch durchführt.
- Konfigurieren Sie anschließend das Routing und die Firewall.
- Führen Sie dann die Integration des Backend- und des Mikrocontrollers durch.
- Dokumentieren Sie die durchgeführten Konfigurationen und die von Ihnen entwickelten Programme.
- Präsentieren Sie ihre Ergebnisse den anderen Projektgruppen.

Aufgabe: Sicherheitsprinzipien und Architekturdesign

- Überprüfen Sie ob das Architekturdesign die fundamentalen Sicherheitsprinzipien erfüllt. Ergänzen Sie aus ihrer Sicht fehlende Eigenschaften.

Sicherheitsprinzip	Bewertung	Ergänzungen
Security-by-Design		
Least Common Mechanism		
Complete Mediation		
Least Privilege		
Fail-Secure Defaults		
Defence in Depth		